

4 Embedded Flash memory (FLASH)

4.1 Introduction

The embedded Flash memory (FLASH) manages the accesses of any master to the 1 Mbyte of embedded non-volatile memory. It implements the read, program and erase operations, error corrections as well as various integrity and confidentiality protection mechanisms.

The embedded Flash memory manages the automatic loading of non-volatile user option bytes at power-on reset, and implements the dynamic update of these options.

4.2 FLASH main features

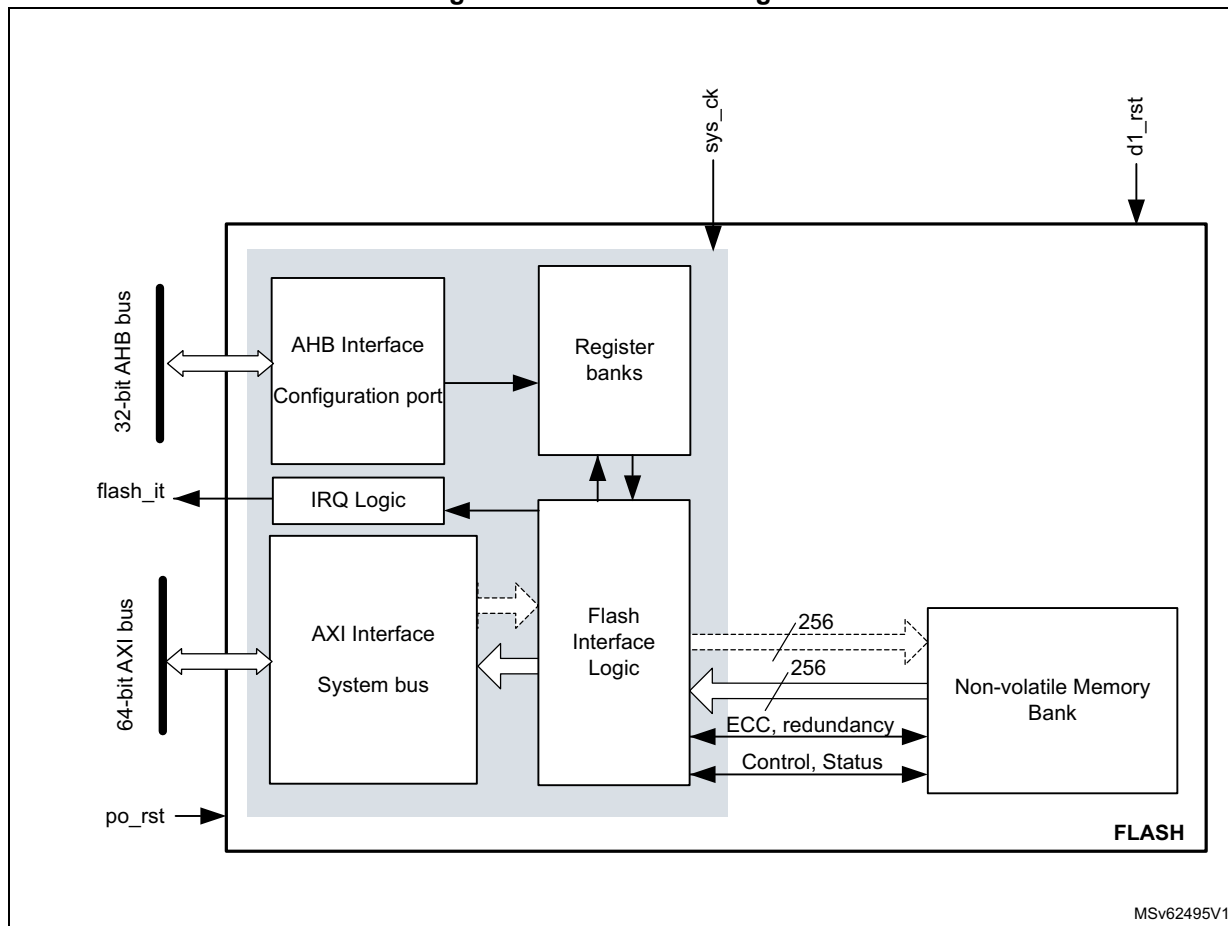
- Up to 1 Mbyte of non-volatile memory (one bank)
- Flash memory read operations supporting multiple length (64 bits, 32bits, 16bits or one byte)
- Flash memory programming by 256 bits
- Sector erase, bank erase
- Error Code Correction (ECC): one error detection/correction or two error detections per 256-bit Flash word using 10 ECC bits
- Cyclic redundancy check (CRC) hardware module
- User configurable non-volatile option bytes
- Flash memory enhanced protections, activated by option bytes
 - Read protection (RDP), preventing unauthorized Flash memory dump to safeguard sensitive application code
 - Sector write-protection (128-Kbytes)
 - One proprietary code readout protection (PCROP) area. When enabled, this area is execute-only.
 - One secure-only area. When enabled this area is accessible only if the STM32 microcontroller operates in Secure access mode.
- Read and write command queues to streamline Flash operations

4.3 FLASH functional description

4.3.1 FLASH block diagram

Figure 5 shows the embedded Flash memory block diagram.

Figure 5. FLASH block diagram



4.3.2 FLASH internal signals

Table 13 describes a list of the useful to know internal signals available at embedded Flash memory level. These signals are not available on the microcontroller pads.

Table 13. FLASH internal input/output signals

Internal signal name	Signal type	Description
sys_ck	Input	D1 domain bus clock (embedded Flash memory AXI interface clock)
po_rst	Input	Power on reset
d1_rst	Input	D1 domain system reset
flash_it	Output	Embedded flash interface interrupt request

4.3.3 FLASH architecture and integration in the system

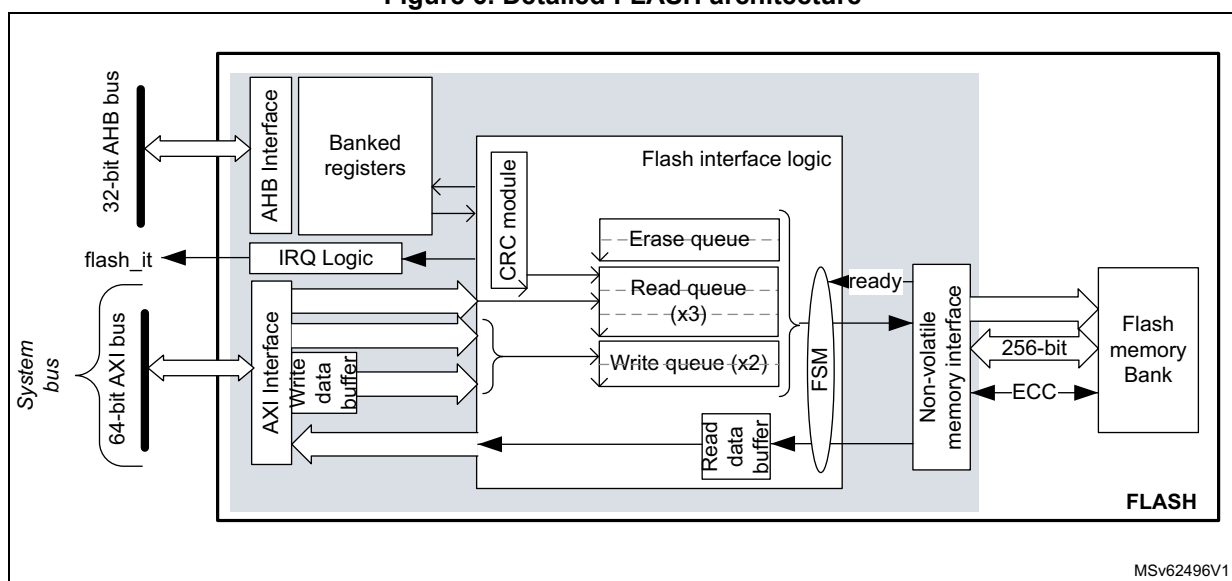
The embedded Flash memory is a central resource for the whole microcontroller. It serves as an interface to one non-volatile memory bank, and organizes the memory in a very specific way. The embedded Flash memory also proposes a set of security features to protect the assets stored in the non-volatile memory at boot time, at run-time and during firmware and configuration upgrades.

The embedded Flash memory offers one 64-bit AXI slave ports for code and data accesses, plus a 32-bit AHB configuration slave port used for register bank accesses.

Note: *The application can simultaneously request a read and a write operation through the AXI interface.*

The embedded Flash microarchitecture is shown in [Figure 6](#).

Figure 6. Detailed FLASH architecture



Behind the system interfaces, the embedded Flash memory implements various command queues and buffers to perform Flash read, write and erase operations with maximum efficiency.

Thanks to the addition of a read and write data buffer, the AXI slave port handles the following access types:

- Multiple length: 64 bits, 32 bits, 16 bits and 8 bits
- Single or burst accesses
- Write wrap burst must not cross 32-byte aligned address boundaries to target exactly one Flash word

The AHB configuration slave port supports 8-bit, 16-bit and 32-bit word accesses.

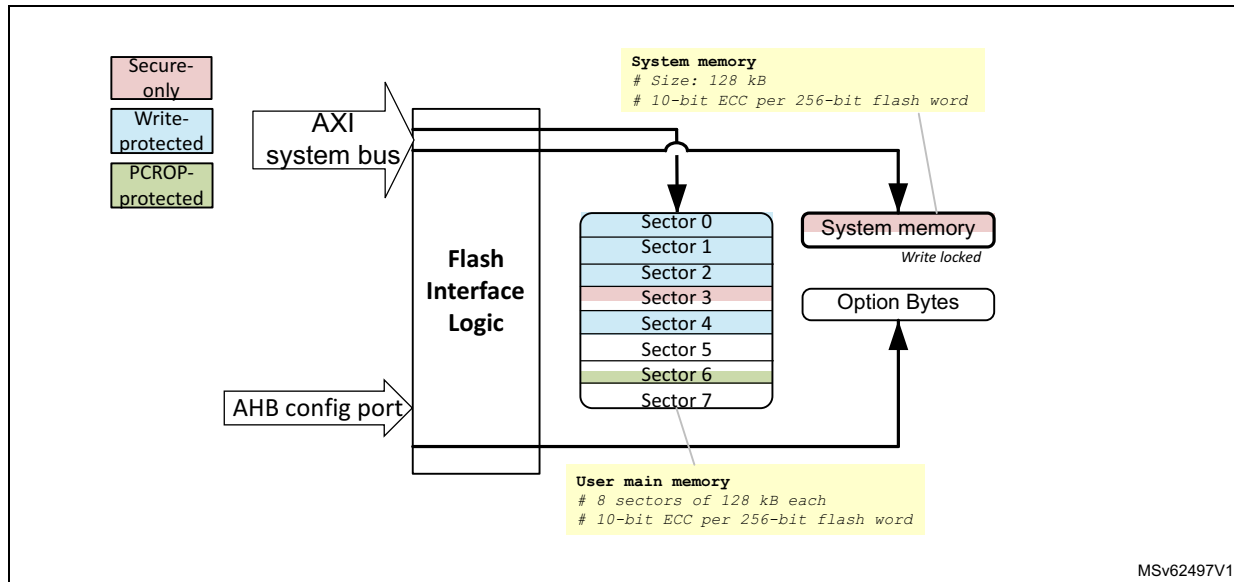
The embedded Flash memory is built in such a way that only one read or write operation can be executed at a time.

4.3.4 Flash memory architecture and usage

Flash memory architecture

Figure 7 shows the non-volatile memory organization supported by the embedded Flash memory.

Figure 7. Embedded Flash memory organization



The embedded Flash non-volatile memory is composed of:

- A memory block of up to 1 Mbyte, with Flash-word rows of 256 bits + 10 bits of ECC per word
- A system memory block of 128 Kbytes
- A special region dedicated to option bytes storage

The embedded Flash memory interface logic supports the following architecture partitioning:

- A user Flash memory mapped to the 1-Mbyte memory block. It is divided into eight sectors of 128 Kbytes. The user Flash memory is ECC protected.
- A system Flash memory mapped to the 128-Kbyte system memory block. The system Flash memory is ECC protected
- A set of non-volatile option bytes loaded at reset by the embedded Flash memory and accessible by the application software only through the AHB configuration register interface.

The overall Flash memory architecture is summarized in [Table 15](#).

Table 14. Flash memory organization (STM32H730 devices)

Flash memory area	Address range	Size (bytes)	Region name	Access interface	SNB ⁽¹⁾
User main memory	0x0800 0000- 0x0801 FFFF	128 K	Sector 0	AXI ports	0x0
System memory	0x1FF0 0000- 0x1FF1 FFFF	128 K	System Flash memory (read-only)		N/A ⁽²⁾
Option bytes	N/A	-	User option bytes	Registers only	N/A

1. SNB contains the target sector number for an erase operation. See [Section 4.3.10](#) for details.

2. Cannot be erased by software.

Table 15. Flash memory organization (STM32H723/733 and STM32H725/735 devices)

Flash memory area	Address range	Size (bytes)	Region name ⁽¹⁾	Access interface	SNB ⁽²⁾
User main memory	0x0800 0000- 0x0801 FFFF	128 K	Sector 0	AXI ports	0x0
	0x0802 0000- 0x0803 FFFF	128 K	Sector 1		0x1
	...	...	...		...
	0x080E 0000- 0x080F FFFF	128 K	Sector 7		0x7
System memory	0x1FF0 0000- 0x1FF1 FFFF	128 K	System Flash memory (read-only)		N/A ⁽³⁾
Option bytes	N/A	-	User option bytes	Registers only	N/A

1. For devices with 512 Kbytes of Flash memory, only sectors 0 to 3 are available.

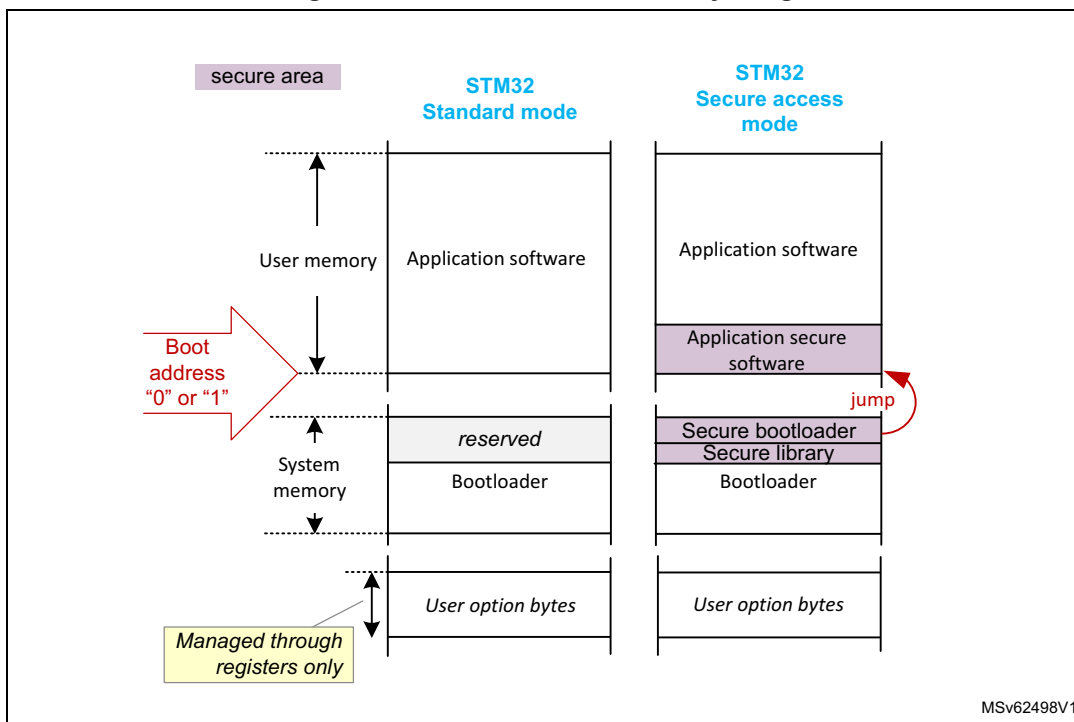
2. SNB contains the target sector number for an erase operation. See [Section 4.3.10](#) for details.

3. Cannot be erased by software.

Partition usage

Figure 8 shows how the embedded Flash memory is used both by STMicroelectronics and the application software.

Figure 8. Embedded Flash memory usage



User and system memories are used differently according to whether the microcontroller is configured by the application software in Standard mode or in Secure access mode. This selection is done through the SECURITY option bit (see [Section 4.4.6](#)):

- In Standard mode, the user memory contains the application code and data, while the system memory is loaded with the STM32 bootloader. When a reset occurs, the core jumps to the boot address configured through the BOOT pin and the BOOT_CM_ADD0/1 option bytes.
- In Secure access mode, dedicated libraries can be used for secure boot. They are located in user Flash and system Flash memory:
 - ST libraries in system Flash memory assist the application software boot with special features such as secure boot and secure firmware install (SFI).
 - Application secure libraries in user Flash memory are used for secure firmware update (SFU).

In Secure access mode, the microcontroller always boots into the secure bootloader code (unique entry point). Then, if no secure services are required, this code securely jumps to the requested boot address configured through the BOOT pin and the option bytes, as shown in [Figure 8](#) (see [Section 5: Secure memory management \(SMM\)](#) for details).

Note: For more information on option byte setup for boot, refer to [Section 4.4.7](#).

Additional partition usage is the following:

- The option bytes are used by STMicroelectronics and by the application software as non-volatile product options (e.g. boot address, protection configuration and reset behaviors).

Note: For further information on STM32 bootloader flashing by STMicroelectronics, refer to application note AN2606 “STM32 microcontroller system memory boot mode” available from <http://www.st.com>.

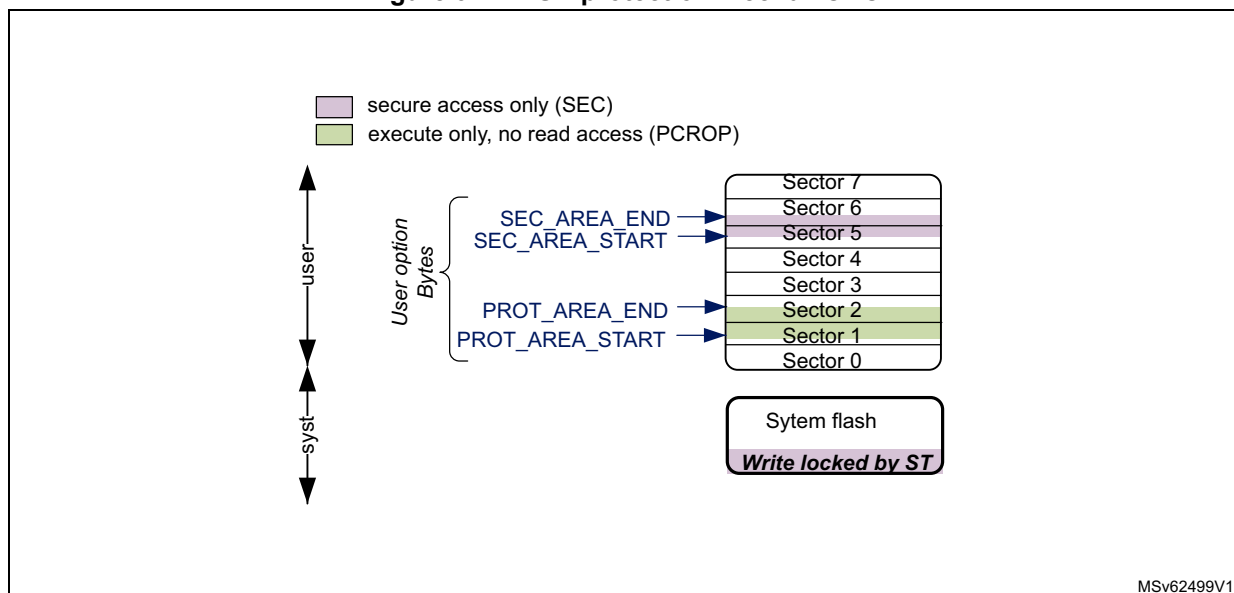
4.3.5 FLASH system performance enhancements

The embedded Flash memory uses read and write command queues in order to enhance Flash operations.

4.3.6 FLASH data protection schemes

[Figure 9](#) gives an overview of the protection mechanisms supported by the embedded Flash memory. A PCROP and a secure-only area can be defined in the Flash memory bank. The properties of these protected areas are detailed in [Section 4.5](#).

Figure 9. FLASH protection mechanisms



4.3.7 Overview of FLASH operations

Read operations

The embedded Flash memory can perform read operations on the whole non-volatile memory using various granularities: 64 bits, 32 bits, 16 bits or one byte. User and system Flash memories are read through the AXI interface, while the option bytes are read through the register interface.

To increase efficiency, the embedded Flash memory implements the buffering of consecutive read requests.

For more details on read operations, refer to [Section 4.3.8: FLASH read operations](#).

Program/erase operations

The embedded Flash memory supports the following program and erase operations:

- Single Flash word write (256-bit granularity), with the possibility for the application to force-write a user Flash word with less than 256 bits
- Single sector erase
- Bank erase
- Option byte update

Note: Program and erase operations are subject to the various protection that could be set on the embedded Flash memory, such as write protection and global readout protection (see next sections for details).

To increase efficiency, the embedded Flash memory implements the buffering of consecutive write accesses.

For more details refer to [Section 4.3.9: FLASH program operations](#) and [Section 4.3.10: FLASH erase operations](#).

Protection mechanisms

The embedded Flash memory supports different protection mechanisms:

- Global readout protection (RDP)
- Proprietary code readout protection (PCROP)
- Write protection
- Secure access only protection

For more details refer to [Section 4.5: FLASH protection mechanisms](#).

Option byte loading

Under specific conditions, the embedded Flash memory reliably loads the non-volatile option bytes stored in non-volatile memory, thus enforcing boot and security options to the whole system when the embedded Flash memory becomes functional again. For more details refer to [Section 4.4: FLASH option bytes](#).

4.3.8 FLASH read operations

Read operation overview

The embedded Flash memory supports the execution of one read command while two are waiting in the read command queue. Multiple read access types are also supported as defined in [Section 4.3.3: FLASH architecture and integration in the system](#).

The read commands are associated with a 256-bit read data buffer.

Note: The embedded Flash memory can perform single error correction and double error detection while read operations are being executed (see [Section 4.3.11: Flash memory error protections](#)).

The AXI interface read channel operates as follows:

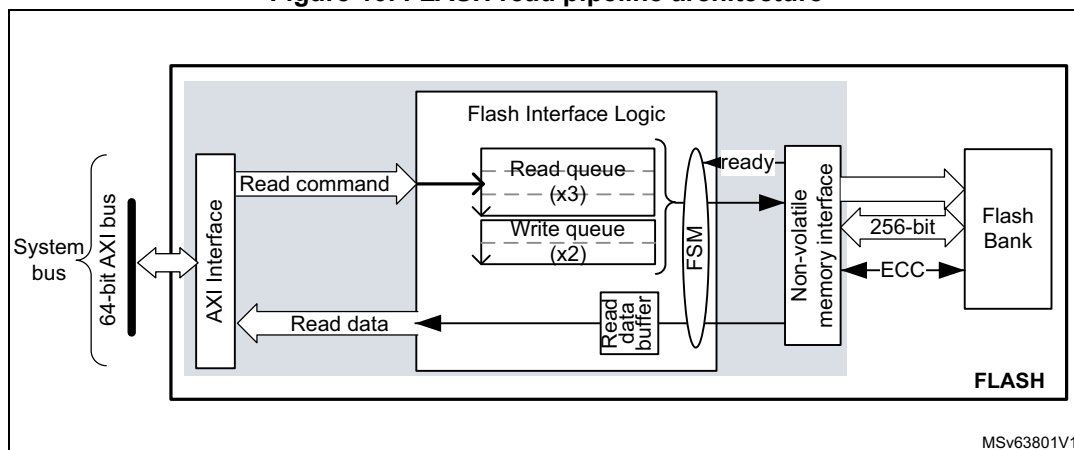
- When the read command queue is full, any new AXI read request stalls the bus read channel interface and consequently the master that issued that request.
- If several consecutive read accesses request data that belong to the same Flash data word (256 bits), the data are read directly from the current data read buffer, without

triggering additional Flash read operations. This mechanism occurs each time a read access is granted. When a read access is rejected for security reasons (e.g. PCROP protected word), the corresponding read error response is issued by the embedded Flash memory and no read operation to Flash memory is triggered.

The Read pipeline architecture is summarized in [Figure 10](#).

For more information on bus interfaces, refer to [Section 4.3.3: FLASH architecture and integration in the system](#).

Figure 10. FLASH read pipeline architecture



Single read sequence

The recommended simple read sequence is the following:

1. Freely perform read accesses to any AXI-mapped area.
2. The embedded Flash memory effectively executes the read operation from the read command queue buffer as soon as the non-volatile memory is ready and the previously requested operations have been served.

Adjusting read timing constraints

The embedded Flash memory clock must be enabled and running before reading data from non-volatile memory.

To correctly read data from Flash memory, the number of wait states (LATENCY) must be correctly programmed in the Flash access control register (FLASH_ACR) according to the embedded Flash memory AXI interface clock frequency (sys_ck) and the internal voltage range of the device (V_{core}).

[Table 16](#) shows the correspondence between the number of wait states (LATENCY), the programming delay parameter (WRHIGHFREQ), the embedded Flash memory clock frequency and its supply voltage ranges.

Table 16. FLASH recommended number of wait states and programming delay

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V _{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;35 MHz]	[0 MHz;50 MHz]	[0 MHz;67 MHz]	[0 MHz;70 MHz]
1 WS (2 FLASH clock cycles)	01	]35 MHz;70 MHz]	]50 MHz;100 MHz]	]67 MHz;133 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	10	]70 MHz;85 MHz]	]100 MHz;150 MHz]	]133 MHz;200 MHz]	]140 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	11	-	-	-	]210 MHz;275MHz]

Adjusting system frequency

After power-on, a default 7 wait-state latency is specified in FLASH_ACR register, in order to accommodate AXI interface clock frequencies with a safety margin (see [Table 16](#)).

When changing the AXI bus frequency, the application software must follow the below sequence in order to tune the number of wait states required to access the non-volatile memory.

To increase the embedded Flash memory clock source frequency:

1. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in the FLASH_ACR register, as described in [Table 16](#).
2. Check that the new number of wait states is taken into account by reading back the FLASH_ACR register.
3. Modify the embedded Flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of the reset and clock controller (RCC).
4. Check that the new embedded Flash memory clock source and/or the new AXI bus clock prescaler value are taken into account by reading back the embedded Flash memory clock source status and/or the AXI bus prescaler value in the RCC_CFGR register of the reset and clock controller (RCC).

To decrease the embedded Flash memory clock source frequency:

1. Modify the embedded Flash memory clock source and/or the AXI bus clock prescaler in the RCC_CFGR register of reset and clock controller (RCC).
2. Check that the embedded Flash memory new clock source and/or the new AXI bus clock prescaler value are taken into account by reading back the embedded Flash

memory clock source status and/or the AXI interface prescaler value in the RCC_CFGR register of reset and clock controller (RCC).

3. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in FLASH_ACR register, as described in [Table 16](#).
4. Check that the new number of wait states has been taken into account by reading back the FLASH_ACR register.

Error code correction (ECC)

The embedded Flash memory embeds an error correction mechanism. Single error correction and double error detection are performed for each read operation. For more details, refer to [Section 4.3.11: Flash memory error protections](#).

Read errors

When the ECC mechanism is not able to correct the read operation, the embedded Flash memory reports read errors as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

Read interrupts

See [Section 4.8: FLASH interrupts](#) for details.

4.3.9 FLASH program operations

Program operation overview

The virgin state of each non-volatile memory bitcell is 1. The embedded Flash memory supports programming operations that can change (reset) any memory bitcell to 0. However these operations do not support the return of a bit to its virgin state. In this case an erase operation of the entire sector is required.

A program operation consists in issuing write commands. The embedded Flash memory supports the execution of one write command while one command is waiting in the write command queue. Since a 10-bit ECC code is associated to each 256-bit data Flash word, only write operations by 256 bits are executed in the non-volatile memory.

Note: The application can decide to write as little as 8 bits to a Flash word. In this case, a force-write mechanism to the 256 bits + ECC is used (see FW bit of FLASH_CR register).

System Flash memory cannot be written by the application software.

It is not recommended to overwrite a Flash word that is not virgin. The result may lead to an inconsistent ECC code that will be systematically reported by the embedded Flash memory, as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

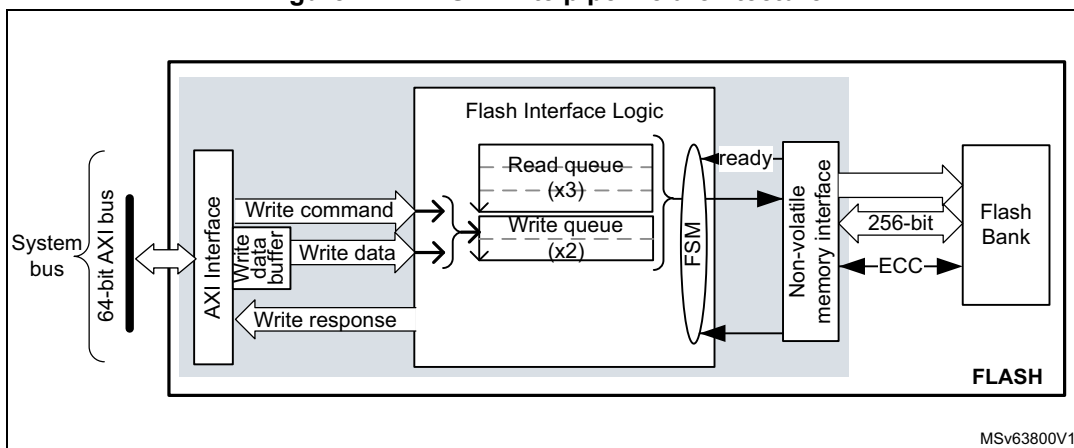
The AXI interface write channel operates as follows:

- A 256-bit write data buffer is associated with the AXI interface. It supports multiple write access types (64 bits, 32 bits, 16 bits and 8 bits).
- When the write queue is full, any new AXI write request stalls the bus write channel interface and consequently the master that issued that request.

The write pipeline architecture is described in [Figure 11](#).

For more information on bus interfaces, refer to [Section 4.3.3: FLASH architecture and integration in the system](#).

Figure 11. FLASH write pipeline architecture



Managing write protections

Before programming a user sector, the application software must check the protection of the targeted Flash memory area.

The embedded Flash memory checks the protection properties of the write transaction target at the output of the write queue buffer, just before the effective write operation to the non-volatile memory:

- If a write protection violation is detected, the write operation is canceled and write protection error (WRPERR) is raised in FLASH_SR register.
- If the write operation is valid, the 10-bit ECC code is concatenated to the 256 bits of data and the write to non-volatile memory is effectively executed.

Note: No write protection check is performed when the embedded Flash memory accepts AXI write requests.

The write protection flag does not need to be cleared before performing a new programming operation.

Monitoring ongoing write operations

The application software can use three status flags located in FLASH_SR in order to monitor ongoing write operations.

- **BSY**: this bit indicates that an effective write, erase or option byte change operation is ongoing to the non-volatile memory.
- **QW**: this bit indicates that a write, erase or option byte change operation is pending in the write queue or command queue buffer. It remains high until the write operation is complete. It supersedes the BSY status bit.
- **WBNE**: this bit indicates that the embedded Flash memory is waiting for new data to complete the 256-bit write buffer. In this state the write buffer is not empty. It is reset as soon as the application software fills the write buffer, force-writes the operation using FW bit in FLASH_CR, or disables all write operations.

Enabling write operations

Before programming the user flash, the application software must make sure that PG bit is set to 1 in FLASH_CR. If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)) and the PG bit must be set.

When the option bytes need to be modified or a mass erase needs to be started, the application software must make sure that FLASH_OPTCR is unlocked. If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)).

Note: *The application software must not unlock a register that is already unlocked, otherwise this register will remain locked until next system reset.*

If needed, the application software can update the programming delay and programming parallelism as described at the end of this section.

Single write sequence

The recommended single write sequence is the following:

1. Unlock the FLASH_CR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
2. Enable write operations by setting the PG bit in the FLASH_CR register.
3. Check the protection of the targeted memory area.
4. Write one Flash-word corresponding to 32-byte data starting at a 32-byte aligned address.
5. Check that QW has been raised and wait until it is reset to 0.

If step 4 is executed incrementally (e.g. byte per byte), the write buffer can become partially filled. In this case the application software can decide to force-write what is stored in the write buffer by using FW bit in FLASH_CR register. In this particular case, the unwritten bits are automatically set to 1. If no bit in the write buffer is set to 0, the FW bit has no effect.

Note: *Using a force-write operation prevents the application from updating later the missing bits with a value different from 1, which is likely to lead to a permanent ECC error.*

Any write access requested while the PG bit is set to 0 is rejected. In this case, no error is generated on the bus, but the PGSERR flag is raised.

Clearing the programming sequence error (PGSERR) and inconsistency error (INCERR) is mandatory before attempting a write operation (see [Section 4.7: FLASH error management](#) for details).

Adjusting programming timing constraints

Program operation timing constraints depend of the embedded Flash memory clock frequency, which directly impacts the performance. If timing constraints are too tight, the non-volatile memory will not operate correctly, if they are too lax, the programming speed will not be optimal.

The user must therefore trim the optimal programming delay through the WRHIGHFREQ parameter in the FLASH_ACR register. Refer to [Table 16](#) in [Section 4.3.8: FLASH read operations](#) for the recommended programming delay depending on the embedded Flash memory clock frequency.

The application software must check that no program/erase operation is ongoing before modifying WRHIGHFREQ parameter.

Adjusting programming parallelism

The parallelism is the maximum number of bits that can be written to 0 in one shot during a write operation. The programming parallelism is also used during sector and bank erase.

There is no hardware limitation on programming parallelism. The user can select different parallelisms depending on the application requirements: the lower the parallelism, the lower the peak consumption during a programming operation, but the longer the execution time.

The parallelism is configured through the PSIZE1/2 bits in FLASH_CR1/2 register.

Table 17. FLASH parallelism parameter

PSIZE	Parallelism
00	8 bits (one byte)
01	16 bits
10	32 bits
11	64 bits

Programming errors

When a program operation fails, an error can be reported as described in [Section 4.7: FLASH error management](#).

Programming interrupts

See [Section 4.8: FLASH interrupts](#) for details.

4.3.10 FLASH erase operations

Erase operation overview

The embedded Flash memory can perform erase operations on 128-Kbyte user sectors.

Note: **System Flash cannot be erased by the application software.**

The erase operation forces all non-volatile bit cells to high state, which corresponds to the virgin state. It clears existing data and corresponding ECC, allowing a new write operation to be performed. If the application software reads back a word that has been erased, all the bits will be read at 1, without ECC error.

Erase operations are similar to read or program operations except that the commands are queued in a special buffer (a two-command deep erase queue).

Erase commands are issued through the AHB configuration interface. If the embedded Flash memory receives simultaneously a write and an erase request, both operations are accepted but the write operation is executed first.

Erase and security

A user sector can be erased only if it does not contain PCROP, secure-only or write-protected data (see [Section 4.5: FLASH protection mechanisms](#) for details). In other words, if the application software attempts to erase a user sector with at least one Flash word that is protected, the sector erase operation is aborted and the WRPERR flag is raised in the FLASH_SR register, as described in [Section 4.7.2: Write protection error \(WRPERR\)](#).

The embedded Flash memory allows the application software to perform an erase followed by an automatic protection removal (PCROP, secure-only area and write protection), as described hereafter.

Enabling erase operations

Before erasing a sector, the application software must make sure that FLASH_CR is unlocked. If it is not the case, an unlock sequence must be used (see [Section 4.5.1: FLASH configuration protection](#)).

Note: *The application software must not unlock a register that is already unlocked, otherwise this register will remain locked until next system reset.*

Similar constraints apply to bank erase requests.

Flash sector erase sequence

To erase a 128-Kbyte user sector, proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock the FLASH_CR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
3. Set the SER bit and SNB bitfield in the corresponding FLASH_CR register. SER indicates a sector erase operation, while SNB contains the target sector number.
4. Set the START bit in the FLASH_CR register.
5. Wait for the QW bit to be cleared in the corresponding FLASH_SR register.

Note: *If a bank erase is requested simultaneously to the sector erase (BER bit set), the bank erase operation supersedes the sector erase operation.*

Standard Flash bank erase sequence

To erase all bank sectors except for those containing secure-only and protected data, proceed as follows:

1. Check and clear (optional) all the error flags due to previous program/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock the FLASH_CR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if the register is not already unlocked).
3. Set the BER bit in the FLASH_CR register.
4. Set the START bit in the FLASH_CR register to start the bank erase operation. Then wait until the QW bit is cleared in the corresponding FLASH_SR register.

Note: *BER and START bits can be set together, so above steps 3 and 4 can be merged.*

If a sector erase is requested simultaneously to the bank erase (SER bit set), the bank erase operation supersedes the sector erase operation.

Flash bank erase with automatic protection-removal sequence

To erase all bank sectors including those containing secure-only and protected data without performing an RDP regression (see [Section 4.5.3](#)), proceed as follows:

1. Check and clear (optional) all the error flags due to previous programming/erase operation. Refer to [Section 4.7: FLASH error management](#) for details.
2. Unlock FLASH_OPTCR register, as described in [Section 4.5.1: FLASH configuration protection](#) (only if register is not already unlocked).
3. If a PCROP-protected area exists set DMEP bit in FLASH_PRAR_PRG register. In addition, program the PCROP area end and start addresses so that the difference is negative, i.e. $PROT_AREA_END < PROT_AREA_START$.
4. If a secure-only area exists set DMES bit in FLASH_SCAR_PRG register. In addition, program the secure-only area end and start addresses so that the difference is negative, i.e. $SEC_AREA_END < SEC_AREA_START$.
5. Set all WRPSn1/2 bits in FLASH_WPSN_PRG to 1 to disable all sector write protection.
6. Unlock FLASH_CR register, only if register is not already unlocked.
7. Set the BER bit in the FLASH_CR register.
8. Set the START bit in the FLASH_CR register to start the bank erase with protection removal operation. Then wait until the QW bit is cleared in the corresponding FLASH_SR register. At that point a bank erase operation has erased the whole bank including the sectors containing PCROP-protected and/or secure-only data, and an option byte change has been automatically performed so that all the protections are disabled.

Note: BER and START bits can be set together, so above steps 8 and 9 can be merged.

Be aware of the following warnings regarding to above sequence:

- No other option bytes than the one indicated above must be changed.
- When the event above occurs, a simple bank erase is done, no option byte change is performed and no option change error is set.

4.3.11 Flash memory error protections

Error correction codes (ECC)

The embedded Flash memory supports an error correction code (ECC) mechanism. It is based on the SECDED algorithm in order to correct single errors and detects double errors.

This mechanism uses 10 ECC bits per 256-bit Flash word, and applies to user and system Flash memory.

More specifically, during each read operation from a 256-bit Flash word, the embedded Flash memory also retrieves the 10-bit ECC information, computes the ECC of the Flash word, and compares the result with the reference value. If they do not match, the corresponding ECC error is raised as described in [Section 4.7.7: Error correction code error \(SNECCERR/DBECCERR\)](#).

During each program operation, a 10-bit ECC code is associated to each 256-bit data Flash word, and the resulting 266-bit Flash word information is written in non-volatile memory.

Cyclic redundancy codes (CRC)

The embedded Flash memory implements a cyclic redundancy check (CRC) hardware module. This module checks the integrity of a given user Flash memory area content (see [Figure 6: Detailed FLASH architecture](#)).

The area processed by the CRC module can be defined either by sectors or by start/end addresses. It can also be defined as the whole bank (user Flash memory area only).

When enabled, the CRC hardware module performs multiple reads by chunks of 4, 16, 64 or 256 consecutive Flash-word (i.e. chunks of 128, 512, 2048 or 8192 bytes). These consecutive read operations are pushed by the CRC module into the required read command queue together with other AXI read requests, thus avoiding to deny AXI read commands.

CRC computation uses CRC-32 (Ethernet) polynomial 0x4C11DB7:

$$X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$

The CRC operation is concurrent with option byte change as the same hardware is used for both operations. To avoid the CRC computation from being corrupted, the application shall complete the option byte change (by reading the result of the change) before running a CRC operation, and vice-versa.

The sequence recommended to configure a CRC operation is the following:

1. Unlock FLASH_CR register, if not already unlocked.
2. Enable the CRC feature by setting the CRC_EN bit in FLASH_CR.
3. Program the desired data size in the CRC_BURST field of FLASH_CRCCR.
4. Define the user Flash area on which the CRC has to be computed. Two solutions are possible:
 - Define the area start and end addresses by programming FLASH_CRCSADDR and FLASH_CRCEADDR, respectively,
 - or select the targeted sectors by setting the CRC_BY_SECT bit in FLASH_CRCCR and by programming consecutively the target sector numbers in the CRC_SECT field of the FLASH_CRCCR register. Set ADD_SECT bit after each CRC_SECT programming.
5. Start the CRC operation by setting the START_CRC bit.
6. Wait until the CRC_BUSY flag is reset in FLASH_SR register.
7. Retrieve the CRC result in the FLASH_CRCDATAR register.

The CRC can be computed for the whole bank by setting the ALL_BANK bit in the FLASH_CRCCR register.

Note: *The application should avoid running a CRC on PCROP- or secure-only user Flash area since it may alter the expected CRC value. A special error flag defined in [Section 4.7.10: CRC read error \(CRCRDERR\)](#) can be used to detect such a case.*

CRC computation does not raise standard read error flags such as RDSERR, RDPERR and DBECCERR. Only CRCRDERR is raised.

4.3.12 FLASH reset and clocks

Reset management

The embedded Flash memory can be reset by a D1 domain reset (d1_rst), driven by the reset and clock control (RCC). The main effects of this reset are the following:

- All registers, except for option byte registers, are cleared, including read and write latencies.
- Most control registers are automatically protected against write operations. To unprotect them, new unlock sequences must be used as described in [Section 4.5.1: FLASH configuration protection](#).

The embedded Flash memory can be reset by a power-on reset (po_rst), driven by the reset and clock control (RCC). When the reset falls, all option byte registers are reset. When the reset rises up, the option bytes are loaded, potentially applying new features. During this loading sequence, the device remains under reset and the embedded Flash memory is not accessible.

The Reset signal can have a critical impact on the embedded Flash memory:

- The contents of the Flash memory are not guaranteed if a device reset occurs during a Flash memory write or erase operation.
- If a reset occurs while the option byte modification is ongoing, the old option byte values are kept. When it occurs, a new option byte modification sequence is required to program the new values.

Clock management

The embedded Flash memory uses the microcontroller system clock (sys_ck), here the AXI interface clock.

Depending on the device clock and internal supply voltage, specific read and write latency settings usually need to be set in the Flash access control register (FLASH_ACR), as explained in [Section 4.3.8: FLASH read operations](#) and [Section 4.3.9: FLASH program operations](#).

4.4 FLASH option bytes

4.4.1 About option bytes

The embedded Flash memory includes a set of non-volatile option bytes. They are loaded at power-on reset and can be read and modified only through configuration registers.

These option bytes are configured by the end-user depending on the application requirements. Some option bytes might have been initialized by STMicroelectronics during manufacturing stage.

This section documents:

- When option bytes are loaded
- How application software can modify them
- What is the detailed list of option bytes, together with their default factory values (i.e. before the first option byte change).

4.4.2 Option byte loading

There are multiple ways of loading the option bytes into embedded Flash memory:

1. Power-on wakeup

When the device is first powered, the embedded Flash memory automatically loads all the option bytes. During the option byte loading sequence, the device remains under reset and the embedded Flash memory cannot be accessed.

2. Wakeup from system Standby

When the D1 power domain, which contains the embedded Flash memory, is switched from DStandby mode to DRun mode, the embedded Flash memory behaves as during a power-on sequence.

3. Ad-hoc option byte reloading by the application

When the user application successfully modifies the option byte content through the embedded Flash memory registers, the non-volatile option bytes are programmed and the embedded Flash memory automatically reloads all option bytes to update the option registers.

Note: The option bytes read sequence is enhanced thanks to a specific error correction code. In case of security issue, the option bytes may be loaded with default values (see [Section 4.4.3: Option byte modification](#)).

4.4.3 Option byte modification

Changing user option bytes

A user option byte change operation can be used to modify the configuration and the protection settings saved in the non-volatile option byte area.

The embedded Flash memory features two sets of option byte registers:

- The first register set contains the current values of the option bytes. Their names have the `_CUR` extension. All “`_CUR`” registers are read-only. Their values are automatically loaded from the non-volatile memory after power-on reset, wakeup from system standby or after an option byte change operation.
- The second register set allows the modification of the option bytes. Their names contain the `_PRG` extension. All “`_PRG`” registers can be accessed in read/write mode.

When the `OPTLOCK` bit in `FLASH_OPTCR` register is set, modifying the `_PRG` registers is not possible.

When `OPTSTART` bit is set to 1, the embedded Flash memory checks if at least one option byte needs to be programmed by comparing the current values (`_CUR`) with the new ones (`_PRG`). If this is the case and all the other conditions are met (see [Changing security option bytes](#)), the embedded Flash memory launches the option byte modification in its non-volatile memory and updates the option byte registers with `_CUR` extension.

If one of the condition described in [Changing security option bytes](#) is not respected, the embedded Flash memory sets the `OPTCHANGEERR` flag to 1 in the `FLASH_OPTSR_CUR` register and aborts the option byte change operation. In this case, the `_PRG` registers are not overwritten by current option value. The user application can check what was wrong in their configuration.

Unlocking the option byte modification

After reset, the OPTLOCK bit is set to 1 and the FLASH_OPTCR is locked. As a result, the application software must unlock the option configuration register before attempting to change the option bytes. The FLASH_OPTCR unlock sequence is described in [Section 4.5.1: FLASH configuration protection](#).

Option byte modification sequence

To modify user option bytes, follow the sequence below:

1. Unlock FLASH_OPTCR register as described in [Section 4.5.1: FLASH configuration protection](#), unless the register is already unlocked.
2. Write the desired new option byte values in the corresponding option registers (FLASH_XXX_PRG).
3. Set the option byte start change OPTSTART bit to 1 in the FLASH_OPTCR register.
4. Wait until OPT_BUSY bit is cleared.

Note: If a reset or a power-down occurs while the option byte modification is ongoing, the original option byte value is kept. A new option byte modification sequence is required to program the new value.

Changing security option bytes

On top of OPTLOCK bit, there is a second level of protection for security-sensitive option byte fields. Specific rules must be followed to update them:

- **Readout protection (RDP)**

A detailed description of RDP option bits is given in [Section 4.5.3](#). The following rules must be respected to modify these option bits:

- When RDP is set to level 2, no changes are allowed. As a result, if the user application attempts to reduce the RDP level, an option byte change error is raised (OPTCHANGEERR bit in FLASH_OPTSR_CUR register), and all the programmed changes are ignored.
- When the RDP is set to level 1, requiring a change to level 2 is always allowed. When requiring a regression to level 0, an option byte change error can occur if some of the recommendations provided in this chapter have not been followed.
- When the RDP is set to level 0, switching to level 1 or level 2 is possible without any restriction.

- **Sector write protection (WRPSn)**

These option bytes manage sector write protection in FLASH_WPSN_CUR1/2R registers. They can be changed without any restriction when the RDP protection level is different from level 2.

- **PCROP area size (PROT_AREA_START and PROT_AREA_END)**

These option bytes configure the size of the PCROP areas in FLASH_PRAR_CUR registers. They can be increased without any restriction by the Arm® Cortex®-M7 core. To remove or reduce a PCROP area, an RDP level 1 to 0 regression (see [Section 4.5.3](#)) or a bank erase with protection removal (see [Section 4.3.10](#)) must be

requested at the same time. DMEP must be set to 1 in either FLASH_PRAR_CUR or FLASH_PRAR_PRG, otherwise an option byte change error is raised.

- **DMEP**

When this option bit is set, the content of the corresponding PCROP area is erased during a RDP level 1 to 0 regression (see [Section 4.5.3](#)) or a bank erase with protection removal (see [Section 4.3.10](#)). It is preserved otherwise.

There are no restrictions in setting DMEP bit. Resetting DMEP bit from 1 to 0 can only be done when an RDP level 1 to 0 regression or a bank erase with protection removal is requested at the same time.

- **Secure access mode (SECURITY)**

The SECURITY option bit activates the secure access mode described in [Section 4.5.5](#). This option bit can be freely set by the application software if such mode is activated on the device. If at least one PCROP or secure-only area is defined as not null, the only way to deactivate the security option bit (from 1 to 0) is to perform an RDP level 1 to 0 regression, when DMEP is set to 1 in either FLASH_PRAR_CUR or FLASH_PRAR_PRG registers, and DMES is set to 1 in either FLASH_SCAR_CUR or FLASH_SCAR_PRG.

If no valid secure-only area and no valid PCROP area are currently defined, the SECURITY option bit can be freely reset.

Note: It is recommended to have both SEC_AREA_START > SEC_AREA_END and PROT_AREA_START > PROT_AREA_END programmed when deactivating the SECURITY option bit during an RDP level 1 to 0 regression.

- **Secure-only area size (SEC_AREA_START and SEC_AREA_END)**

These option bytes configure the size of the secure-only areas in FLASH_SCAR_CUR registers. They can be changed without any restriction by the user secure application or by the ST secure library running on the device. For user non-secure application, the secure-only area size can be removed by performing a bank erase with protection removal (see [Section 4.3.10](#)), or an RDP level 1 to 0 regression when DMES set to 1 in either FLASH_SCAR_CUR or FLASH_SCAR_PRG (otherwise an option byte change error is raised).

- **DMES**

When this option bit is set, the content of the corresponding secure-only area is erased during an RDP level 1 to 0 regression or a bank erase with protection removal, it is preserved otherwise.

DMES bits can be set without any restriction. Resetting DMES bit from 1 to 0 can only be performed when an RDP level 1 to 0 regression or a bank erase with protection removal is requested at the same time.

- **Secure DTCM size (ST_RAM_SIZE)**

These bits selects the size of the secure DTCM, an option that is available only when SECURITY option bit is set. It can be modified only when the CPU is running the ST secure library.

4.4.4 Option bytes overview

[Table 18](#) lists all the user option bytes managed through the embedded Flash memory registers, as well as their default values before the first option byte change (default factory value).

Table 18. Option byte organization

Register	Bitfield															
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FLASH_OPTSR_x[31:16]	Res.	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURITY	ST_RAM_SIZE		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
Default factory value	0	0	0	1	0	1	1	1	1	0	0	1	1	1	1	0
FLASH_OPTSR_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	RDP[7:0]								NRST_STOP_D1	NRST_STOP_D1	Res.	IWDG1_SW	BOR_LEV		Res.	Res.
Default factory value	1	0	1	0	1	0	1	0	1	1	1	1	0	0	0	0
FLASH_OPTSR2_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CPUFREQ_BOOST	TCM_AXI_SHARED	
Default factory value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
FLASH_BOOT_x[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	BOOT_CM_ADD1[15:0]															
Default factory value	0x1FF0															
FLASH_BOOT_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	BOOT_CM_ADD0[15:0]															
Default factory value	0x0800															
FLASH_PRAR_x[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMEP	Res.	Res.	Res.	PROT_AREA_END											

Table 18. Option byte organization (continued)

Register	Bitfield															
Default factory value	0	0	0	0	0x000											
FLASH_PRAR_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	PROT_AREA_START											
Default factory value	0	0	0	0	0x0FF											
FLASH_SCAR_x[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	DMES	Res.	Res.	Res.	SEC_AREA_END											
Default factory value	1	0	0	0	0x000											
FLASH_SCAR_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	SEC_AREA_START											
Default factory value	0	0	0	0	0x0FF											
FLASH_WPSN_x[31:16]	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
Default factory value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
FLASH_WPSN_x[15:0]	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn[7:0]							
Default factory value	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

4.4.5 Description of user and system option bytes

Below the list of the general-purpose option bytes that can be used by the application:

- Watchdog management
 - IWDG_FZ_STOP: independent watchdog (IWDG1) counter active in Stop mode if 1 (stop counting or freeze if 0)
 - IWDG_FZ_SDBY: independent watchdog (IWDG1) counter active in Standby mode if 1 (stop counting or freeze if 0)
 - IWDG1_SW: hardware (0) or software (1) IWDG1 watchdog control selection

Note: If the hardware watchdog “control selection” feature is enabled (set to 0), the watchdog is automatically enabled at power-on, thus generating a reset unless the watchdog key register is written to or the down-counter is reloaded before the end-of-count is reached.

Depending on the configuration of IWDG_STOP and IWDG_STBY options, the IWDG can continue counting (1) or not (0) when the device is in Stop or Standby mode, respectively.

When the IWDG is kept running during Stop or Standby mode, it can wake up the device from these modes.

- Reset management
 - BOR: Brownout level option, indicating the supply level threshold that activates/releases the reset
 - NRST_STDBY_D1: generates a reset when D1 domain enters DStandby mode. It is active low.
 - NRST_STOP_D1: generates a reset when D1 domain enters DStop mode. It is active low.

Note: Whenever a Standby (respectively Stop) mode entry sequence is successfully executed, the device is reset instead of entering Standby (respectively Stop) mode if NRST_STDBY (respectively NRST_STOP) is cleared to 0.

- Device options
 - IO_HSLV: I/O speed optimization at low-voltage if set to 1.

When STMicroelectronics delivers the device, the values programmed in the general-purpose option bytes are the following:

- Watchdog management
 - IWDG1 active in Standby and Stop modes (option byte value = 0x1)
 - IWDG1 not automatically enabled at power-on (option byte value = 0x1)
- Reset management:
 - BOR: brownout level option (reset level) equals brownout reset threshold 0 (option byte value = 0x0)
 - A reset is not generated when D1 domain enters DStandby or DStop low-power mode (option byte value = 0x1)
- Device working in the full voltage range with I/O speed optimization at low-voltage disabled (IO_HSLV=0)

Refer to [Section 4.9: FLASH registers](#) for details.

4.4.6 Description of data protection option bytes

Below the list of the option bytes that can be used to enhance data protection:

- RDP[7:0]: Readout protection level (see [Section 4.5.3 on page 179](#) for details).
- WRPSn: sector write- protection (see [Section 4.5.2 on page 178](#) for details).
- PROT_AREAx: Proprietary code readout protection (refer to [Section 4.5.4 on page 183](#) for details)
 - PROT_AREA_START (respectively PROT_AREA_END) contains the first (respectively last) 256-byte block of the PCROP zone
 - DMEP: when set to 1, the PCROP area is erased during a RDP protection level regression (change from level 1 to 0) or a bank erase with protection removal.
- SEC_AREAx: secure access only zones definition (refer to [Section 4.5.5 on page 184](#) for details).
 - SEC_AREA_START (respectively SEC_AREA_END) contains the first (respectively last) 256-byte block of the secure access only zone
 - DMES: when set to 1 the secure access only zone is erased during a RDP protection level regression (change from level 1 to 0), or a bank erase with protection removal.
- SECURITY: this non-volatile option can be used by the application to manage secure access mode, as described in [Section 4.5.5](#).
- ST_RAM_SIZE: this non-volatile option defines the amount of DTCM RAM root secure services (RSS) can use during execution when the SECURITY bit is set. The DTCM RAM is always fully available for the application whatever the option byte configuration.

When STMicroelectronics delivers the device, the values programmed in the data protection option bytes are the following:

- RDP level 0 (option byte value = 0xAA)
- Flash bank erase operations do not impact secure-only and PCROP data areas when enabled by the application (DMES=DMEP=0)
- PCROP and secure-only zone protections disabled (start addresses higher than end addresses)
- Write protection disabled (all option byte bits set to 1)
- Secure access mode disabled (SECURITY option byte value= 0)
- RSS can use the full DTCM RAM for executing its services (ST_RAM_SIZE = 11)

Refer to [Section 4.9: FLASH registers](#) for details.

4.4.7 Description of boot address option bytes

Below the list of option bytes that can be used to configure the appropriate boot address for your application:

- BOOT_CM_ADD0/1: MSB of the Arm® Cortex®-M7 boot address when BOOT pin is low (respectively high)

When STMicroelectronics delivers the device, the values programmed in the boot address option bytes are the following:

- Arm® Cortex®-M7 boot address (MSB): 0x0800 (BOOT pin low for user Flash memory) and 0x1FF0 (BOOT pin high for System Flash memory)

Refer to [Section 4.9: FLASH registers](#) for details.

4.5 FLASH protection mechanisms

Since sensitive information can be stored in the Flash memory, it is important to protect it against unwanted operations such as reading confidential areas, illegal programming of protected area, or illegal Flash memory erasing.

The embedded Flash memory implements the following protection mechanisms that can be used by end-user applications to manage the security of embedded non-volatile storage:

- Configuration protection
- Global device Readout protection (RDP)
- Write protection
- Proprietary code readout protection (PCROP)
- Secure access mode areas

This section provides a detailed description of all these security mechanisms.

4.5.1 FLASH configuration protection

The embedded Flash memory uses hardware mechanisms to protect the following assets against unwanted or spurious modifications (e.g. software bugs):

- Option bytes change
- Write operations
- Erase commands
- Interrupt masking

More specifically, write operations to embedded Flash memory control registers (FLASH_CR and FLASH_OPTCR) are not allowed after reset.

The following sequence must be used to unlock FLASH_CR register:

1. Program KEY1 to 0x45670123 in FLASH_KEYR key register.
2. Program KEY2 to 0xCDEF89AB in FLASH_KEYR key register.
3. LOCK bit is now cleared and FLASH_CR is unlocked.

The following sequence must be used to unlock FLASH_OPTCR register:

1. Program OPTKEY1 to 0x08192A3B in FLASH_OPTKEYR option key register.
2. Program OPTKEY2 to 0x4C5D6E7F in FLASH_OPTKEYR option key register.
3. OPTLOCK bit is now cleared and FLASH_OPTCR register is unlocked.

Any wrong sequence locks up the corresponding register/bit until the next system reset, and generates a bus error.

The FLASH_CR (respectively FLASH_OPTCR) register can be locked again by software by setting the LOCK bit in FLASH_CR register (respectively OPTLOCK bit in FLASH_OPTCR register).

In addition the FLASH_CR register remains locked and a bus error is generated when the following operations are executed:

- programming a third key value
- writing to a different register than FLASH_KEYR before FLASH_CR has been completely unlocked (KEY1 programmed but KEY2 not yet programmed)
- writing less than 32 bits to KEY1 or KEY2.

Similarly the FLASH_OPTCR register remains locked and a bus error is generated when the following operations are executed:

- programming a third key value
- writing to a different register before FLASH_OPTCR has been completely unlocked (OPTKEY1 programmed but OPTKEY2 not yet programmed)
- writing less than 32 bits to OPTKEY1 or OPTKEY2.

The embedded Flash memory configuration registers protection is summarized in [Table 19](#).

Table 19. Flash interface register protection summary

Register Name	Unlocking register	Protected asset
FLASH_ACR	N/A	-
FLASH_KEYR	N/A	-
FLASH_OPTKEYR	N/A	-
FLASH_CR	FLASH_KEYR	Write operations Erase commands Interrupt generation masking sources
FLASH_SR	N/A	-
FLASH_CCR	N/A	-
FLASH_OPTCR	FLASH_OPTKEYR	Option bytes change Mass erase
FLASH_OPTSR_PRG FLASH_OPTSR2_PRG	FLASH_OPTCR	Option bytes change
FLASH_OPTCCR	N/A	-
FLASH_PRAR_PRG	FLASH_OPTCR	Option bytes (PCROP)
FLASH_SCAR_PRG	FLASH_OPTCR	Option bytes (security)
FLASH_WPSN_PRG	FLASH_OPTCR	Option bytes (write protection)
FLASH_BOOT_PRG	FLASH_OPTCR	Option bytes (boot)
FLASH_CRCCR	N/A	-
FLASH_CRCSADDR		-
FLASH_CRCEADDR		-
FLASH_CRCDATAR		-
FLASH_ECC_FAR	N/A	-

4.5.2 Write protection

The purpose of embedded Flash memory write protection is to protect the embedded Flash memory against unwanted modifications of the non-volatile code and/or data.

Any 128-Kbyte Flash sector can be independently write-protected or unprotected by clearing/setting the corresponding WRPSn bit in the FLASH_WPSN_PRG register.

A write-protected sector can neither be erased nor programmed. As a result, a bank erase cannot be performed if one sector is write-protected, unless a bank erase with automatic

protection removal or an RDP level 1 to 0 regression is executed (see [Section : Flash bank erase with automatic protection-removal sequence](#) for details).

The embedded Flash memory write-protection user option bits can be modified without any restriction when the RDP level is set to level 0 or level 1. When it is set to level 2, the write protection bitfield can no more be changed in the option bytes.

Note: PCROP or secure-only areas are write and erase protected.

Write protection errors are documented in [Section 4.7: FLASH error management](#).

4.5.3 Readout protection (RDP)

The embedded Flash memory readout protection is global as it does not apply only to the embedded Flash memory, but also to the other secured regions. This is done by using dedicated security signals.

In this section *other secured regions* are defined as:

- Backup SRAM
- RTC backup registers
- Encrypted regions protected by on-the-fly decryption engine (OTFDEC)

The global readout protection level is set by writing the values given in [Table 20](#) into the readout protection (RDP) option byte (see [Section 4.4.3: Option byte modification](#)).

Table 20. RDP value vs readout protection level

RDP option byte value	Global Readout Protection Level
0xAA	Level 0
0xCC	Level 2
Any other value	Level 1 ⁽¹⁾

1. Default protection level when RDP option byte is erased.

Definitions of RDP global protection level

RDP Level 0 (no protection)

When the global read protection level 0 is set, all read/program/erase operations from/to the user Flash memory are allowed (if no others protections are set). This is true whatever the boot configuration (boot from user or system flash memory, boot from RAM), and whether the debugger is connected to the device or not. Accesses to the *other secured regions* are also allowed.

RDP Level 1 (Flash memory content protection)

When the global read protection level 1 is set, the below properties apply:

- The embedded flash memory content is protected against debugger and potential malicious code stored in RAM. Hence as soon as any debugger is connected or has been connected, or a boot is configured in embedded RAM (intrusion), the embedded Flash memory prevents any accesses to Flash memory.
- When no intrusion is detected (no boot in RAM, no boot in System Flash memory and no debugger connected), all read/program/erase operations from/to the user Flash

memory are allowed (if no others protections are set). Accesses to the *other secured regions* are also allowed.

- When an intrusion is detected, no accesses to the user Flash memory can be performed. A bus error is generated when a read access is requested to the Flash memory. In addition, no accesses to *other secured regions* (read or write) can be performed.
- When performing an RDP level regression, i.e. programming the RDP protection to level 0, the user Flash memory and the *other secured regions* are mass erased, as described in [RDP protection transitions](#).
- When booting from STMicroelectronics non-secure bootloader, only the identification services are available (GET_ID_COMMAND, GET_VER_COMMAND and GET_CMD_COMMAND).

RDP Level 2 (device protection and intrusion prevention)

When the global read protection level 2 is set, the below rules apply:

- All debugging features are disabled.
- Like level 0, all read/write/erase operations from/to the user Flash memory are allowed since the debugger and the boot from RAM and System Flash memory are disabled. Accesses to the *other secured regions* are also allowed.
- Booting from RAM is no more allowed.
- The user option bits described in [Section 4.4](#) can no longer be changed.

Caution: Memory read protection level 2 is an irreversible operation. When level 2 is activated, the level of protection cannot be changed back to level 0 or level 1.

Note: *The JTAG port is permanently disabled when level 2 is active (acting as a JTAG fuse). As a consequence, STMicroelectronics is not able to perform analysis on defective parts on which the level 2 protection has been set.*

Apply a power-on reset if the global read protection level 2 is set while the debugger is still connected.

The above RDP global protection is summarized in [Table 21](#).

Table 21. Protection vs RDP Level⁽¹⁾

Boot area	Inputs		Effects				Comment
	RDP	Debugger connected	User Flash memory access ⁽²⁾	System Flash memory access ⁽³⁾	Other secured regions	Option Bytes access	
User Flash memory	Level 0	Yes ⁽⁴⁾ /No	R/W/E	R	R/W	R/W	-
	Level 1	Yes ⁽³⁾	illegal access ⁽⁵⁾	R	no access	R/W	
	Level 1	No	R/W/E	R	R/W	R/W	
	Level 2	No	R/W/ E	R	R/W	R	

Table 21. Protection vs RDP Level⁽¹⁾ (continued)

Boot area	Inputs		Effects				Comment
	RDP	Debugger connected	User Flash memory access ⁽²⁾	System Flash memory access ⁽³⁾	Other secured regions	Option Bytes access	
RAM or System Flash memory	Level 0	Yes ⁽³⁾ /No	R/W/E	R	R/W	R/W	-
	Level 1	Yes ⁽³⁾ /No	illegal access ⁽⁵⁾	R	no access	R/W	When selected, only ST basic bootloader is executed
	Level 2	No	Not applicable				No boot from RAM or ST system Flash memory in RDP level 2

1. R = read, W = write, E = erase.

2. PCROP (see [Section 4.5.4](#)) and secure-only access control (see [Section 4.5.5](#)) applies.

3. Read accesses to secure boot and secure libraries stored in system Flash possible only from STMicroelectronics code.

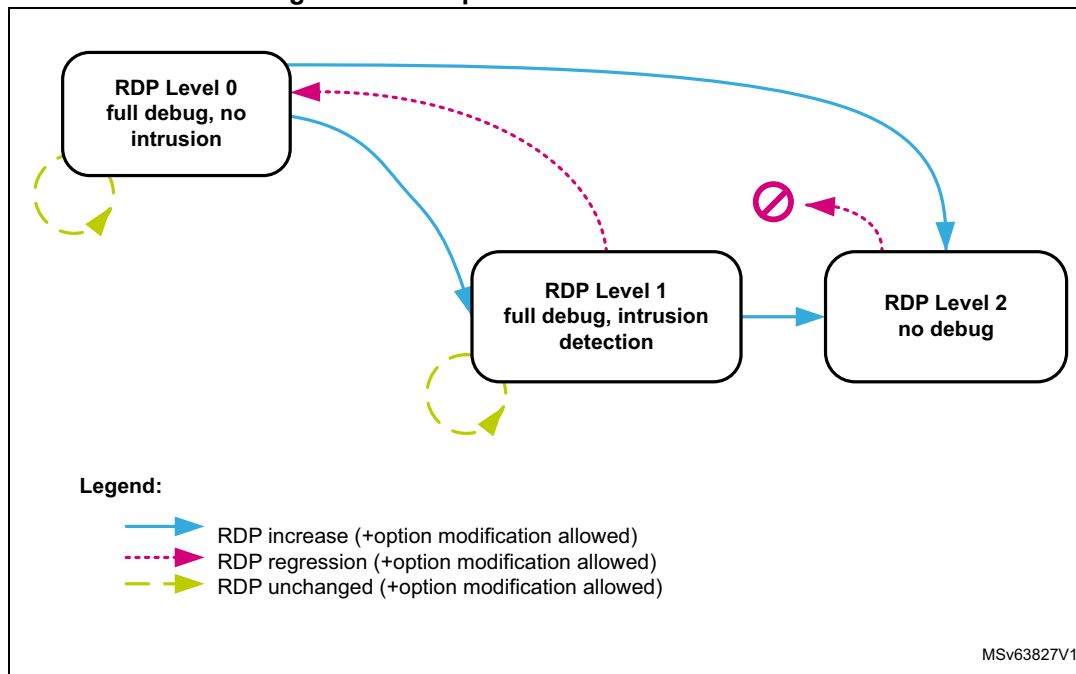
4. JTAG interface disabled while secure libraries are executed.

5. [Read protection error \(RDPErr\)](#) with bus error on read operations, [Write protection error \(WRPErr\)](#) on write/erase operations.

RDP protection transitions

[Figure 12](#) shows how to switch from one RDP level to another. The transition is effective after successfully writing the option bytes including RDP (refer to [Section 4.4.3](#) for details on how to change the option bytes).

Figure 12. RDP protection transition scheme



[Table 22](#) details the RDP transitions and their effects on the product.

Table 22. RDP transition and effects

RDP transition		RDP option update	Effect on device		
Level before change	Level after change		Debugger disconnect	Option bytes change	Mass erase
L0	L1	not 0xAA and not 0xCC	No	Allowed	No
	L2	0xCC	Yes	Not allowed	No
L1	L2	0xCC	Yes	Not allowed	No
	L0	0xAA	No	Allowed	Yes
L0	L0	0xAA	No	Allowed	No
L1	L1	not 0xAA and not 0xCC			

When the current RDP level is RDP level 1, requesting a new RDP level 0 causes a full mass erase:

- The user Flash memory area of the embedded Flash memory is mass erased:
 - A partial sector erase occurs if PCROP (respectively secure-only) areas are preserved by the application. It happens when both DMES bits (respectively DMES bits) are set to 0 in FLASH_PRAR_CUR and FLASH_PRAR_PRG (respectively FLASH_SCAR_CUR and FLASH_SCAR_PRG). The sectors belonging to the preserved area(s) are not erased.
 - A full bank erase occurs when at least one DMES bit is set to 1 in FLASH_PRAR_CUR or FLASH_PRAR_PRG, and at least one DMES bit is set to 1 in FLASH_SCAR_CUR or FLASH_SCAR_PRG.

Note: Data in write protection area are not preserved during RDP regression.

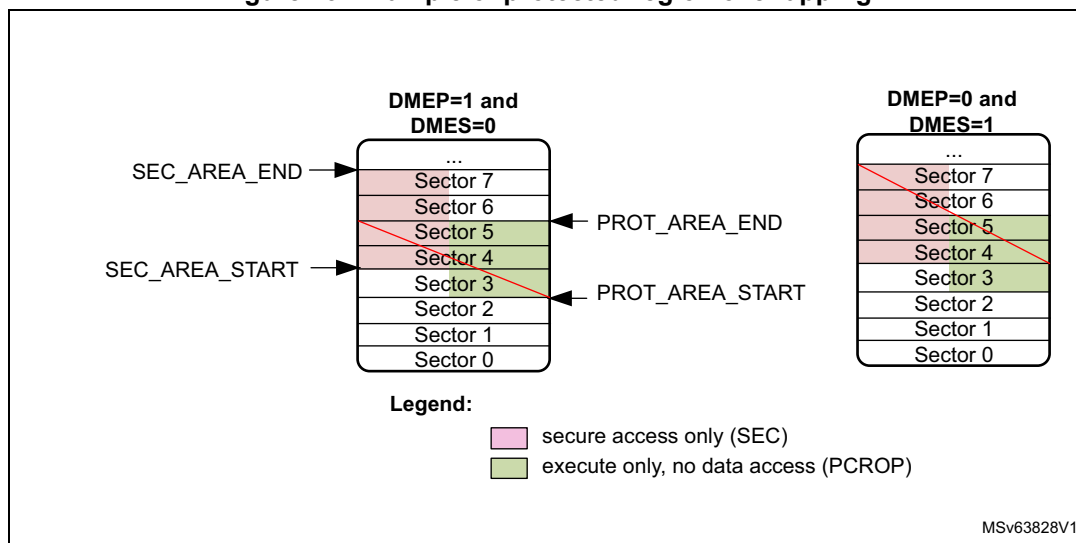
- The other secured regions are also erased.

During a level regression, if a PCROP area overlaps with a secure-only area, the embedded Flash memory performs the erase operation depending on the DMES/DMEP options bits (see strike-through areas in red in [Figure 13](#)). More specifically:

- When DMEP is set in FLASH_PRAR_CUR or FLASH_PRAR_PRG, the PCROP area is erased (overlapped or not with secure-only area).
- When DMES is set in FLASH_SCAR_CUR or FLASH_SCAR_PRG, the secure-only area is erased (overlapped or not with PCROP area).

Note: The sector protections (PCROP, secure-only) are removed only if the protected sector boundaries are modified by the user application.

Figure 13. Example of protected region overlapping



About RDP protection errors

Whatever the RDP level, the corresponding error flag is raised when an illegal read or write access is detected (see [Section 4.7: FLASH error management](#)).

4.5.4 Proprietary code readout protection (PCROP)

The embedded Flash memory allows the definition of an “executable-only” area in the user area. In this area, only instruction fetch transactions from the system, i.e. no data access (data or literal pool) are allowed. This protection is particularly efficient to protect third party software intellectual property.

Note: Executable-only area usage requires the native code to be compiled accordingly using “execute-only” option.

PCROP area programming

One PCROP area can be defined by setting the PROT_AREA_END and PROT_AREA_START option bytes so that the END address is strictly higher than the START address. PROT_AREA_START and PROT_AREA_END are defined with a granularity of 256 bytes. This means that the actual PCROP area size (in bytes) is defined by:

$$[(\text{PROT_AREA_END} - \text{PROT_AREA_START}) + 1] \times 256$$

As an example, to set a PCROP area on the first 4 Kbytes (i.e. from address 0x0800 0000 to address 0x0800 0FFF, both included), the embedded Flash memory must be configured as follows:

- PROT_AREA_START[11:0] = 0x000
- PROT_AREA_END[11:0] = 0x00F

The protected area size defined above is equal to:

$$[(\text{PROT_AREA_END} - \text{PROT_AREA_START}) + 1] \times 256 = 16 \times 256 \text{ bytes} = 4 \text{ Kbytes.}$$

The minimum execute-only PCROP area that can be set is 16 Flash words (or 512 bytes). The maximum area is the whole user Flash memory, configured by setting to the same value the PCROP area START and END addresses.

Note: It is recommended to align the PCROP area size with the Flash sector granularity in order to avoid access right issues.

PCROP area properties

Each valid PCROP area has the following properties:

- Arm® Cortex®-M7 debug events are ignored while executing code in this area.
- Only the CPU can access it (Master ID filtering), using only instruction fetch transactions. In all other cases, accessing the PCROP area is illegal (see below).
- Illegal transactions to a PCROP area (i.e. data read or write, not fetch) are managed as below:
 - Read operations return a zero, write operations are ignored.
 - No bus error is generated but an error flag is raised (RDPERR for read, WRPERR for write).
- A valid PCROP area is erase-protected. As a result:
 - No erase operations to a sector located in this area is possible (including the sector containing the area start address and the end address)
 - No bank erase can be performed if a valid PCROP area is defined, except during regression level or erase with protection removal.
- Only the CPU can modify the PCROP area definition and DMEP bit, as explained in [Changing user option bytes](#) in [Section 4.4.3](#).
- During an RDP level 1 to 0 regression where the PCROP area is not null
 - The PCROP area content is not erased if the DMEP bits are both set to 0 in FLASH_PRAR_CUR and FLASH_PRAR_PRG registers.
 - The PCROP area content is erased if either of the corresponding DMEP bit is set to 1 in FLASH_PRAR_CUR or FLASH_PRAR_PRG register.

For more information on PCROP protection errors, refer to [Section 4.7: FLASH error management](#).

4.5.5 Secure access mode

The embedded Flash memory allows the definition of a secure-only area in the user area. This area can be accessed only while the CPU executes secure application code. This feature is available only if the SECURITY option bit is set to 1.

Secure-only areas help isolating secure user code from application non-secure code. As an example, they can be used to protect a customer secure firmware upgrade code, a custom secure boot library or a third party secure library.

Secure-only area programming

One secure-only area can be defined by setting the SEC_AREA_END and SEC_AREA_START option bytes so that the END address is strictly higher than the START address. SEC_AREA_START and SEC_AREA_END are defined with a granularity of 256 bytes. This means that the actual secure-only area size (in bytes) is defined by:

$$[(\text{SEC_AREA_END} - \text{SEC_AREA_START}) + 1] \times 256$$

As an example, to set a secure-only area on the first 8 Kbytes (i.e. from address 0x0800 0000 to address 0x0800 1FFF, both included), the embedded Flash memory must be configured as follows:

- SEC_AREA_START[11:0] = 0x000
- SEC_AREA_END[11:0] = 0x01F

The secure-only area size defined above is equal to:

$$[(\text{SEC_AREA_END} - \text{SEC_AREA_START}) + 1] \times 256 = 32 \times 256 \text{ bytes} = 8 \text{ Kbytes.}$$

Note: These option bytes can be modified only by the CPU running ST security library or application secure code, except during regression level or erase with protection removal.

The minimum secure-only area that can be set is 16 Flash words (or 512 bytes). The maximum area is the whole user Flash memory bank, configured by setting to the same value the secure-only area START and END addresses.

Note: It is recommended to align the secure-only area size with Flash sector granularity in order to avoid access right issues.

Secure-only access area properties

- Arm® Cortex®-M7 debug events are ignored while executing code in this area.
- Only the CPU executing ST secure library or user secure application can access it (Master ID filtering). In all other cases, accessing the secure-only area is illegal (see below).
- Illegal transactions to a secure-only area are managed as follows:
 - Data read transactions return zero. Data write transactions are ignored. No bus error is generated but an error flag is raised (RDSERR for read, WRPERR for write).
 - Read instruction transactions generate a bus error and the RDSERR error flag is raised.
- A valid secure-only area is erase-protected. As a result:
 - No erase operations to a sector located in this area are possible (including the sector containing the area start address and the end address), unless the application software is executed from a valid secure-only area.
 - No bank erase can be performed if a valid secure-only area is defined, except during regression level or erase with protection removal.
- Only the CPU can modify the secure-only area definition and DMES bit, as explained in [Changing user option bytes](#) in [Section 4.4.3](#).
- During an RDP level 1 to 0 regression where the secure-only area is not null:
 - the secure-only area content is not erased if the DMES bits are both set to 0 in FLASH_SCAR_CUR and FLASH_SCAR_PRG registers.
 - the secure-only area content is erased if either of the DMES bits is set to 1 in FLASH_SCAR_CUR or FLASH_SCAR_PRG register.

For more information on secure-only protection errors, refer to [Section 4.7: FLASH error management](#).

4.6 FLASH low-power modes

4.6.1 Introduction

The table below summarizes the behavior of the embedded Flash memory in the microcontroller low-power modes. The embedded Flash memory belongs to the D1 domain.

Table 23. Effect of low-power modes on the embedded Flash memory

System state	Power mode		D1 domain voltage range	Allowed if FLASH busy	FLASH power mode (in D1 domain)
	D1 domain	D2 domain			
Run	DRun	DRun, DStop or DStandby	VOS0/1/2/3	Yes	Run
	DStop	DStop or DStandby	VOS0/1/2/3	No	Off
	DStandby	DStop or DStandby	Off	No	Off
Stop	DStop	DStop or DStandby	SVOS3/4/5	No	Clock gated or Stopped
	DStandby	DStop or DStandby	Off	No	Off
Standby ⁽¹⁾	DStandby	DStandby		No	

1. D3 domain must always be in DStandby mode. When all clocks are stopped and the CPU is in CStop, the VCore domain is switched off.

When the system state changes or within a given system state, the embedded Flash memory might get a different voltage supply range (VOS) according to the application. The procedure to switch the embedded Flash memory into various power mode (run, clock gated, stopped, off) is described hereafter.

Note: For more information in the microcontroller power states, refer to the Power control section (PWR).

4.6.2 Managing the FLASH domain switching to DStop or DStandby

As explain in [Table 23](#), if the embedded Flash memory informs the reset and clock controller (RCC) that it is busy (i.e. BSY, QW, WBNE is set), the microcontroller cannot switch the D1 domain to DStop or DStandby mode.

Note: CRC_BUSY is not taken into account.

There are two ways to release the embedded Flash memory:

- Reset the WBNE busy flag in FLASH_SR register by any of the following actions:
 - a) Complete the write buffer with missing data.
 - b) Force the write operation without filling the missing data by activating the FW bit in FLASH_CR register. This forces all missing data “high”.
 - c) Reset the PG bit in FLASH_CR register. This disables the write buffer and consequently lead to the loss of its content.
- Poll QW busy bits in FLASH_SR register until they are cleared. This will indicate that all recorded write, erase and option change operations are complete.

The microcontroller can then switch the domain to DStop or DStandby mode.

4.7 FLASH error management

4.7.1 Introduction

The embedded Flash memory automatically reports when an error occurs during a read, program or erase operation. A wide range of errors are reported:

- [Write protection error \(WRPERR\)](#)
- [Programming sequence error \(PGSERR\)](#)
- [Strobe error \(STRBERR\)](#)
- [Inconsistency error \(INCERR\)](#)
- [Operation error \(OPERR\)](#)
- [Error correction code error \(SNECCERR/DBECCERR\)](#)
- [Read protection error \(RDPERR\)](#)
- [Read secure error \(RDSERR\)](#)
- [CRC read error \(CRCDERR\)](#)
- [Option byte change error \(OPTCHANGEERR\)](#)

The application software can individually enable the interrupt for each error, as detailed in [Section 4.8: FLASH interrupts](#).

Note: For some errors, the application software must clear the error flag before attempting a new operation.

4.7.2 Write protection error (WRPERR)

When an illegal erase/program operation is attempted to the non-volatile memory bank, the embedded Flash memory sets the write protection error flag WRPERR in FLASH_SR register.

An erase operation is rejected and flagged as illegal if it targets one of the following memory areas:

- A sector belonging to a valid PCROP area (even partially)
- A sector belonging to a valid secure-only area (even partially) except if the application software is executed from a valid secure-only area
- A sector write-locked with WRPSn

An program operation is ignored and flagged as illegal if it targets one of the following memory areas:

- A user Flash sector belonging to a valid PCROP area while the device is not executing an ST secure library
- A user Flash sector belonging to a valid secure-only area while the device is not executing user secure code or ST secure library
- A user sector write-locked with WRPSn
- The system Flash memory
- The user main Flash memory when RDP level is 1, a debugger has been detected on the device, or the CPU has not booted from user Flash memory.
- A reserved area

When WRPERR flag is raised, the operation is rejected and nothing is changed in the bank. If a write burst operation was ongoing, WRPERR is raised each time a Flash word write operation is processed by the embedded Flash memory.

Note: **WRPERR flag does not block any new erase/program operation.**

Not resetting the write protection error flag (WRPERR) does not generate a PGSERR error.

WRPERR flag is cleared by setting CLR_WRPERR bit to 1 in FLASH_CCR register.

If WRPERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when WRPERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.3 Programming sequence error (PGSERR)

When the programming sequence is incorrect, the embedded Flash memory sets the programming sequence error flag PGSERR in FLASH_SR register.

More specifically, PGSERR flag is set if one of below conditions is met:

- A write operation is requested but the program enable bit (PG) has not been set in FLASH_CR register prior to the request.
- The inconsistency error (INCERR) has not been cleared to 0 before requesting a new write or erase operation.

When PGSERR flag is raised, the current program operation is aborted and nothing is changed. The write data buffer is also flushed. If a write burst operation was ongoing, PGSERR is raised at the end of the burst.

Note: **When PGSERR flag is raised, there is a risk that the last write operation performed by the application has been lost because of the above protection mechanism. Hence it is recommended to generate interrupts on PGSERR and verify in the interrupt handler if the last write operation has been successful by reading back the value in the Flash memory.**

The PGSERR flag also blocks any new program operation. This means that PGSERR must be cleared before starting a new program operation.

PGSERR flag is cleared by setting CLR_PGSERR bit to 1 in FLASH_CCR register.

If PGSERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when PGSERR flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.4 Strobe error (STRBERR)

When the application software writes several times to the same byte write buffer, the embedded Flash memory sets the strobe error flag STRBERR in FLASH_SR register.

When STRBERR flag is raised, the current program operation is not aborted. The application can ignore the error, proceed with the current write operation and request new write operations. If a write burst was ongoing, STRBERR is raised at the end of the burst.

STRBERR flag is cleared by setting CLR_STRBERR bit to 1 in FLASH_CCR register.

If STRBERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when STRBERR flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.5 Inconsistency error (INCERR)

When a programming inconsistency is detected, the embedded Flash memory sets the inconsistency error flag INCERR in register FLASH_SR.

More specifically, INCERR flag is set when one of the following conditions is met:

- A write operation is attempted before completion of the previous write operation, e.g.
 - The application software starts a write operation to fill the 256-bit write buffer, but sends a new write burst request to a different Flash memory address before the buffer is full.
 - One master starts a write operation, but before the buffer is full, another master starts a new write operation to the same address or to a different address.
- A wrap burst request issued by a master overlaps two or more 256-bit Flash-word addresses, i.e. wrap bursts must be done within 256-bit Flash-word address boundaries.

Note: INCERR flag must be cleared before starting a new write operation, otherwise a sequence error (PGSERR) is raised.

It is recommended to follow the sequence below to avoid losing data when an inconsistency error occurs:

1. Execute a handler routine when INCERR flag is raised.
2. Stop all write requests to embedded Flash memory.
3. Verify that the write operations that have been requested just before the INCERR event have been successful by reading back the programmed values from the memory.
4. Clear the corresponding INCERR bit.
5. Restart the write operations where they have been interrupted.

INCERR flag is cleared by setting CLR_INCERR bit to 1 in FLASH_CCR register.

If INCERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when INCERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.6 Operation error (OPERR)

When an error occurred during a write or an erase operation, the embedded Flash memory sets the operation error flag OPERR in FLASH_SR register. This error may be caused by an incorrect non-volatile memory behavior due to cycling issues or to a previous modify operation stopped by a system reset.

When OPERR flag is raised, the current program/erase operation is aborted.

OPERR flag is cleared by setting CLR_OPERR bit to 1 in FLASH_CCR register.

If OPERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when OPERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.7 Error correction code error (SNECCERR/DBECCERR)

When a single error correction is detected during a read the embedded Flash memory sets the single error correction flag SNECCERR in FLASH_SR register.

When two ECC errors are detected during a read, the embedded Flash memory sets the double error detection flag DBECCERR in FLASH_SR register. When SNECCERR flag is raised, the corrected read data are returned. Hence the application can ignore the error and request new read operations.

If a read burst operation was ongoing, SNECCERR or DBECCERR flag is raised each time a new data is sent back to the requester through the AXI interface.

When SNECCERR or DBECCERR flag is raised, the address of the Flash word that generated the error is saved in the FLASH_ECC_FAR register. This register is automatically cleared when the associated flag that generated the error is reset.

Note: In case of successive single correction or double detection errors, only the address corresponding to the first error is stored in FLASH_ECC_FAR register.

When DBECCERR flag is raised, a bus error is generated. In case of successive double error detections, a bus error is generated each time a new data is sent back to the requester through the AXI interface.

Note: It is not mandatory to clear SNECCERR or DBECCERR flags before starting a new read operation.

SNECCERR (respectively DBECCERR) flag is cleared by setting to 1 CLR_SNECCERR bit in FLASH_CCR register.

If SNECCERR (respectively DBECCERR) bit in FLASH_CR register is set to 1, an interrupt is generated when SNECCERR (respectively DBECCERR) flag is raised. See [Section 4.8: FLASH interrupts](#) for details.

4.7.8 Read protection error (RDPERR)

When a read operation to a PCROP, a secure-only or a RDP protected area is attempted in non-volatile memory bank, the embedded Flash memory sets the read protection error flag RDPERR in FLASH_SR register.

When RDPERR flag is raised, the current read operation is aborted but the application can request new read operations. If a read burst was ongoing, RDPERR is raised each time a data is sent back to the requester through the AXI interface.

Note: A bus error is raised if a standard application attempts to execute on a secure-only or a RDP protected area.

RDPERR flag is cleared by setting CLR_RDPERR bit to 1 in FLASH_CCR register.

If RDPERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when RDPERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.9 Read secure error (RDSERR)

When a read operation is attempted to a secure address, the embedded Flash memory sets the read secure error flag RDSERR in FLASH_SR register. For more information, refer to [Section 4.5.5: Secure access mode](#).

When RDSERR flag is raised, the current read operation is aborted and the application can request new read operations. If a read burst was ongoing, RDSERR is raised each time a data is sent back to the requester through the AXI interface.

Note: The bus error is raised only if the illegal access is due to an instruction fetch.

RDSERR flag is cleared by setting CLR_RDSERR bit to 1 in FLASH_CCR register.

If RDSERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when RDSERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.10 CRC read error (CRCDERR)

After a CRC computation, the embedded Flash memory sets the CRC read error flag CRCDERR in FLASH_SR register when one or more address belonging to a protected area was read by the CRC module. A protected area corresponds to a PCROP area (see [Section 4.5.4](#)) or to a secure-only area (see [Section 4.5.5](#)).

CRCDERR flag is raised when CRCEND bit is set to 1 (end of CRC calculation). In this case, it is likely that the CRC result is wrong since illegal read operations to protected areas return null values.

CRCDERR flag is cleared by setting CLR_CRCDERR bit to 1 in FLASH_CCR register.

If CRCDERRIE bit in FLASH_CR register is set to 1, an interrupt is generated when CRCDERR flag is raised together with CRCEND bit (see [Section 4.8: FLASH interrupts](#) for details).

4.7.11 Option byte change error (OPTCHANGEERR)

When the embedded Flash memory finds an error during an option change operation, it aborts the operation and sets the option byte change error flag OPTCHANGEERR in FLASH_OPTSR_CUR register.

OPTCHANGEERR flag is cleared by setting CLR_OPTCHANGEERR bit to 1 in FLASH_OPTCCR register.

If OPTCHANGEERRIE bit in FLASH_OPTCR register is set to 1, an interrupt is generated when OPTCHANGEERR flag is raised (see [Section 4.8: FLASH interrupts](#) for details).

4.7.12 Miscellaneous HardFault errors

The following events generate a bus error on the corresponding bus interface:

- On AXI system bus:
 - accesses to user Flash memory while RDP is set to 1 and a illegal condition is detected (boot from system Flash memory, boot from RAM, or debugger connected)
 - fetching to secure-only user Flash memory without the correct access rights
- On AHB configuration bus:
 - wrong key input to FLASH_KEYR or FLASH_OPTKEYR

4.8 FLASH interrupts

The embedded Flash memory can generate a maskable interrupt to signal the following events:

- Read and write errors (see [Section 4.7: FLASH error management](#))
 - Single ECC error correction during read operation
 - Double ECC error detection during read operation
 - Write inconsistency error
 - Bad programming sequence
 - Strobe error during write operations
 - Non-volatile memory write/erase error (due to cycling issues)
 - option change operation error
- Security errors (see [Section 4.7: FLASH error management](#))
 - Write protection error
 - Read protection error
 - Read secure error
 - CRC computation on PCROP or secure-only area error
- Miscellaneous events (described below)
 - End of programming
 - CRC computation complete

These multiple sources are combined into a single interrupt signal, **flash_it**, which is the only interrupt signal from the embedded Flash memory that drives the NVIC (nested vectored interrupt controller).

You can individually enable or disable embedded Flash memory interrupt sources by changing the mask bits in the FLASH_CR register. Setting the appropriate mask bit to 1 enables the interrupt.

Note: Prior to writing, FLASH_CR register must be unlocked as explained in [Section 4.5.1: FLASH configuration protection](#)

[Table 24](#) gives a summary of the available embedded Flash memory interrupt features. As mentioned in the table below, some flags need to be cleared before a new operation is triggered.

Table 24. Flash interrupt request

Interrupt event	Event flag	Enable control bit	Clear flag to resume operation	Bus error
End-of-program event	EOP	EOPIE	N/A	N/A
CRC complete event	CRCEND	CRCENDIE	N/A	N/A
Write protection error	WRPERR	WRPERRIE	No	No
Programming sequence error	PGSERR	PGSERRIE	Yes ⁽¹⁾	No
Strobe error	STRBERR	STRBERRIE	No	No
Inconsistency error	INCERR	INCERRIE	Yes ⁽¹⁾	No
Operation error	OPERR	OPERRIE	No	No

Table 24. Flash interrupt request (continued)

Interrupt event	Event flag	Enable control bit	Clear flag to resume operation	Bus error
ECC single error correction event	SNECCERR	SNECCERRIE	No	No
ECC double error detection event	DBECCERR	DBECCERRIE	No	Yes
Read protection error	RDPERR	RDPERRIE	No	Yes ⁽²⁾
Read secure error	RDSERR	RDSERRIE	No	No (data) Yes (fetch)
CRC read error	CRCRDERR	CRCRDERRIE	No	No
Option Bytes operation error	OPTCHANGEERR	OPTCHANGEERRIE	No	No

1. Programming still possible on the AXI interface that did not trigger the inconsistency error, on the Flash bank that does not have the flag bit raised. See [Section 4.7.5: Inconsistency error \(INCERR\)](#) for details.
2. Bus error occurs only when accessing RDP protected area, as defined in [Section 4.5.3: Readout protection \(RDP\)](#)

The status of the individual maskable interrupt sources described in [Table 24](#) (except for option byte error) can be read from the FLASH_SR register. They can be cleared by setting to 1 the adequate bit in FLASH_CCR register.

Note: No unlocking mechanism is required to clear an interrupt.

End-of-program event

Setting the end-of-operation interrupt enable bit (EOPIE) in the FLASH_CR register enables the generation of an interrupt at the end of an erase operation, a program operation or an option byte change. The EOP bit in the FLASH_SR register is also set when one of these events occurs.

Setting CLR_EOP bit to 1 in FLASH_CCR register clears EOP flag.

CRC end of calculation event

Setting the CRC end-of-calculation interrupt enable bit (CRCENDIE) in the FLASH_CR register enables the generation of an interrupt at the end of a CRC operation. The CRCEND bit in the FLASH_SR register is also set when this event occurs.

Setting CLR_CRCEND bit to 1 in FLASH_CCR register clears CRCEND flag.

4.9 FLASH registers

4.9.1 FLASH access control register (FLASH_ACR)

Address offset: 0x000

Reset value: 0x0000 0037

For more details, refer to [Section 4.3.8: FLASH read operations](#) and [Section 4.3.9: FLASH program operations](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRHIGHFREQ [1:0]		LATENCY[3:0]			
										rw	rw	rw	rw	rw	rw

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:4 **WRHIGHFREQ[1:0]**: Flash signal delay

These bits are used to control the delay between non-volatile memory signals during programming operations. The application software has to program them to the correct value depending on the embedded Flash memory interface frequency. Please refer to [Table 16](#) for details.

Note: No check is performed by hardware to verify that the configuration is correct.

Bits 3:0 **LATENCY[3:0]**: Read latency

These bits are used to control the number of wait states used during read operations. The application software has to program them to the correct value depending on the embedded Flash memory interface frequency and voltage conditions.

0000: zero wait state used to read a word from non-volatile memory

0001: one wait state used to read a word from non-volatile memory

0010: two wait states used to read a word from non-volatile memory

...

1111: 15 wait states used to read from non-volatile memory

Note: No check is performed by hardware to verify that the configuration is correct.

4.9.2 FLASH key register (FLASH_KEYR)

Address offset: 0x004

Reset value: 0x0000 0000

FLASH_KEYR is a write-only register. The following values must be programmed consecutively to unlock FLASH_CR register:

- 1st key = 0x4567 0123
- 2nd key = 0xCDEF 89AB

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **KEY[31:0]**: Non-volatile memory bank configuration access unlock key

4.9.3 FLASH option key register (FLASH_OPTKEYR)

Address offset: 0x008

Reset value: 0x0000 0000

FLASH_OPTKEYR is a write-only register. The following values must be programmed consecutively to unlock FLASH_OPTCR register:

- 1st key = 0x0819 2A3B
- 2nd key = 0x4C5D 6E7F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OPTKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **OPTKEY[31:0]**: FLASH option bytes control access unlock key

4.9.4 FLASH control register (FLASH_CR)

Address offset: 0x00C

Reset value: 0x0000 0031

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERRIE	CRCENDIE	DBECCERRIE	SNECCERRIE	RDSEERRIE	RDPERRIE	OPERRIE	INCERRIE	Res.	STRBERRIE	PGSEERRIE	WRPERRIE	EOPIE
			rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_LEN	Res.	Res.	Res.	Res.	SNB[2:0]			START	FW	PSIZE[1:0]		BER	SER	PG	LOCK
rw					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rs

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CRCRDERRIE**: CRC read error interrupt enable bit

When CRCRDERRIE bit is set to 1, an interrupt is generated when a protected area (PCROP or secure-only) has been detected during the last CRC computation. CRCRDERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a CRC read error occurs

1: interrupt generated when a CRC read error occurs

Bit 27 **CRCENDIE**: CRC end of calculation interrupt enable bit

When CRCENDIE bit is set to 1, an interrupt is generated when the CRC computation has completed. CRCENDIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when CRC computation complete

1: interrupt generated when CRC computation complete

Bit 26 **DBECCERRIE**: ECC double detection error interrupt enable bit

When DBECCERRIE bit is set to 1, an interrupt is generated when an ECC double detection error occurs during a read operation. DBECCERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when an ECC double detection error occurs

1: interrupt generated if an ECC double detection error occurs

Bit 25 **SNECCERRIE**: ECC single correction error interrupt enable bit

When SNECCERRIE bit is set to 1, an interrupt is generated when an ECC single correction error occurs during a read operation. SNECCERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when an ECC single correction error occurs

1: interrupt generated when an ECC single correction error occurs

Bit 24 **RDSERRIE**: Secure error interrupt enable bit

When RDSERRIE bit is set to 1, an interrupt is generated when a secure error (access to a secure-only protected address) occurs during a read operation. RDSERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a secure error occurs

1: an interrupt is generated when a secure error occurs

Bit 23 **RDPERRIE**: Read protection error interrupt enable bit

When RDPERRIE bit is set to 1, an interrupt is generated when a read protection error occurs (access to an address protected by PCROP or by RDP level 1) during a read operation. RDPERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a read protection error occurs

1: an interrupt is generated when a read protection error occurs

Bit 22 **OPERRIE**: Write/erase error interrupt enable bit

When OPERRIE bit is set to 1, an interrupt is generated when an error is detected during a write/erase operation. OPERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a write/erase error occurs

1: interrupt generated when a write/erase error occurs

Bit 21 **INCERRIE**: Inconsistency error interrupt enable bit

When INCERRIE bit is set to 1, an interrupt is generated when an inconsistency error occurs during a write operation. INCERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a inconsistency error occurs

1: interrupt generated when a inconsistency error occurs.

Bit 20 Reserved, must be kept at reset value.

Bit 19 STRBERRIE: Strobe error interrupt enable bit

When STRBERRIE bit is set to 1, an interrupt is generated when a strobe error occurs (the master programs several times the same byte in the write buffer) during a write operation. STRBERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a strobe error occurs

1: interrupt generated when strobe error occurs.

Bit 18 PGSERRIE: Programming sequence error interrupt enable bit

When PGSERRIE bit is set to 1, an interrupt is generated when a sequence error occurs during a program operation. PGSERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a sequence error occurs

1: interrupt generated when sequence error occurs.

Bit 17 WRPERRIE: Write protection error interrupt enable bit

When WRPERRIE bit is set to 1, an interrupt is generated when a protection error occurs during a program operation. WRPERRIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated when a protection error occurs

1: interrupt generated when a protection error occurs.

Bit 16 EOPIE: End-of-program interrupt control bit

Setting EOPIE bit to 1 enables the generation of an interrupt at the end of a program operation. EOPIE can be programmed only when LOCK1 is set to 0.

0: no interrupt generated at the end of a program operation.

1: interrupt enabled when at the end of a program operation.

Bit 15 CRC_EN: CRC control bit

Setting CRC_EN bit to 1 enables the CRC calculation. CRC_EN does not start CRC calculation but enables CRC configuration through FLASH_CRCCR register.

When CRC calculation is performed, it can only be disabled by setting CRC_EN bit to 0.

Resetting CRC_EN clears CRC configuration and resets the content of FLASH_CRCDATAR register.

Clearing CRC_EN to 0 sets CRCDATA to 0x0.

CRC_EN can be programmed only when LOCK1 is set to 0.

Bits 14:11 Reserved, must be kept at reset value.

Bits 10:8 SNB[2:0]: Sector erase selection number

These bits are used to select the target sector for a sector erase operation (they are unused otherwise). SNB can be programmed only when LOCK is set to 0.

000: sector 0

001: sector 1

...

111: sector 7

Bit 7 START: Erase start control bit

START bit is used to start a sector erase or a bank erase operation. START can be programmed only when LOCK is set to 0.

The embedded Flash memory resets START when the corresponding operation has been acknowledged. The user application cannot access any embedded Flash memory register until the operation is acknowledged.

Bit 6 **FW**: Write forcing control bit

FW forces a write operation even if the write buffer is not full. In this case all bits not written are set to 1 by hardware. FW can be programmed only when LOCK is set to 0.

The embedded Flash memory resets FW when the corresponding operation has been acknowledged. The user application cannot access any embedded Flash memory register until the operation is acknowledged.

Note: Using a force-write operation prevents the application from updating later the missing bits with something else than 1, because it is likely that it will lead to permanent ECC error.

Write forcing is effective only if the write buffer is not empty (in particular, FW does not start several write operations when the force-write operations are performed consecutively).

Bits 5:4 **PSIZE[1:0]**: program size

PSIZE selects the parallelism used by the non-volatile memory during write and erase operations. PSIZE can be programmed only when LOCK is set to 0.

00: programming executed with byte parallelism

01: programming executed with half-word parallelism

10: programming executed with word parallelism

11: programming executed with double word parallelism

Bit 3 **BER**: erase request

Setting BER bit to 1 requests a bank erase operation (user Flash memory only). BER can be programmed only when LOCK is set to 0.

BER has a higher priority than SER: if both are set, the embedded Flash memory executes a bank erase.

0: bank erase not requested

1: bank erase requested

Note: Write protection error is triggered when a bank erase is required and some sectors are protected.

Bit 2 SER: sector erase request

Setting SER bit to 1 requests a sector erase. SER can be programmed only when LOCK is set to 0.

BER has a higher priority than SER: if both bits are set, the embedded Flash memory executes a bank erase.

0: sector erase not requested

1: sector erase requested

Note: Write protection error is triggered when a sector erase is required on protected sector(s).

Bit 1 PG: internal buffer control bit

Setting PG bit to 1 enables internal buffer for write operations. This allows preparing program operations even if a sector or bank erase is ongoing.

PG can be programmed only when LOCK is set to 0. When PG is reset, the internal buffer is disabled for write operations, and all the data stored in the buffer but not sent to the operation queue are lost.

Bit 0 LOCK: configuration lock bit

This bit locks the FLASH_CR register. The correct write sequence to FLASH_KEYR register unlocks this bit. If a wrong sequence is executed, or if the unlock sequence to FLASH_KEYR is performed twice, this bit remains locked until the next system reset.

LOCK can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK changes from 0 to 1, the other bits of FLASH_CR register do not change.

0: FLASH_CR register unlocked

1: FLASH_CR register locked

4.9.5 FLASH status register (FLASH_SR)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CRCRDERR	CRCEND	DBECCERR	SNECCERR	RDSERR	RDPEER	OPERR	INCERR	Res.	STRBERR	PGSERR	WRPERR	EOP
			r	r	r	r	r	r	r	r		r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BSY	QW	WBNE	BSY
												r	r	r	r

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 CRCRDERR: CRC read error flag

CRCRDERR flag is raised when a word is found read protected during a CRC operation. An interrupt is generated if CRCRDIE and CRCEND are set to 1. Writing 1 to CLR_CRCRDERR bit in FLASH_CCR register clears CRCRDERR.

0: no protected area detected inside address read by CRC

1: a protected area has been detected inside address read by CRC. CRC result is very likely incorrect.

Note: This flag is valid only when CRCEND bit is set to 1

Bit 27 CRCEND: CRC end of calculation flag

CRCEND bit is raised when the CRC computation has completed. An interrupt is generated if CRCENDIE is set to 1. It is not necessary to reset CRCEND before restarting CRC computation. Writing 1 to CLR_CRCEND bit in FLASH_CCR register clears CRCEND.

0: CRC computation not complete

1: CRC computation complete

Bit 26 DBECCERR: ECC double detection error flag

DBECCERR flag is raised when an ECC double detection error occurs during a read operation. An interrupt is generated if DBECCERRIE is set to 1. Writing 1 to CLR_DBECCERR bit in FLASH_CCR register clears DBECCERR.

0: no ECC double detection error occurred

1: ECC double detection error occurred

Bit 25 SNECCERR: Single correction error flag

SNECCERR flag is raised when an ECC single correction error occurs during a read operation. An interrupt is generated if SNECCERRIE is set to 1. Writing 1 to CLR_SNECCERR bit in FLASH_CCR register clears SNECCERR.

0: no ECC single correction error occurs

1: ECC single correction error occurs

Bit 24 RDSERR: Secure error flag

RDSERR flag is raised when a read secure error (read access to a secure-only protected word) occurs. An interrupt is generated if RDSERRIE is set to 1. Writing 1 to CLR_RDSERR bit in FLASH_CCR register clears RDSERR.

0: no secure error occurs

1: a secure error occurs

Bit 23 RDPERR: Read Protection error flag

RDPERR flag is raised when an read protection error (read access to a PCROP-protected or a RDP-protected area) occurs. An interrupt is generated if RDPERRIE is set to 1. Writing 1 to CLR_RDPERR bit in FLASH_CCR register clears RDPERR.

0: no read protection error occurs

1: a read protection error occurs

Bit 22 OPERR: Write/erase error flag

OPERR flag is raised when an error occurs during a write/erase. An interrupt is generated if OPERRIE is set to 1. Writing 1 to CLR_OPERR bit in FLASH_CCR register clears OPERR.

0: no write/erase error occurs

1: a write/erase error occurs

Bit 21 INCERR: Inconsistency error flag

INCERR flag is raised when a inconsistency error occurs. An interrupt is generated if INCERRIE is set to 1. Writing 1 to CLR_INCERR bit in the FLASH_CCR register clears INCERR.

0: no inconsistency error occurs

1: a inconsistency error occurs

Bit 20 Reserved, must be kept at reset value.**Bit 19 STRBERR:** Strobe error flag

STRBERR flag is raised when a strobe error occurs (when the master attempts to write several times the same byte in the write buffer). An interrupt is generated if the STRBERRIE bit is set to 1. Writing 1 to CLR_STRBERR bit in FLASH_CCR register clears STRBERR.

0: no strobe error occurs

1: a strobe error occurs

Bit 18 PGSERR: Programming sequence error flag

PGSERR flag is raised when a sequence error occurs. An interrupt is generated if the PGSERRIE bit is set to 1. Writing 1 to CLR_PGSERR bit in FLASH_CCR register clears PGSERR.

0: no sequence error occurs

1: a sequence error occurs

Bit 17 WRPERR: Write protection error flag

WRPERR flag is raised when a protection error occurs during a program operation. An interrupt is also generated if the WRPERRIE is set to 1. Writing 1 to CLR_WRPERR bit in FLASH_CCR register clears WRPERR.

0: no write protection error occurs

1: a write protection error occurs

Bit 16 EOP: End-of-program flag

EOP flag is set when a programming operation completes. An interrupt is generated if the EOPIE is set to 1. It is not necessary to reset EOP before starting a new operation. EOP bit is cleared by writing 1 to CLR_EOP bit in FLASH_CCR register.

0: no programming operation completed

1: a programming operation completed

Bits 15:4 Reserved, must be kept at reset value.

Bit 3 **CRC_BUSY**: CRC busy flag

CRC_BUSY flag is set when a CRC calculation is ongoing. This bit cannot be forced to 0. The user must wait until the CRC calculation has completed or disable CRC computation.

0: no CRC calculation ongoing
1: CRC calculation ongoing

Bit 2 **QW**: Wait queue flag

QW flag is set when a write, erase or option byte change operation is pending in the command queue buffer. It is not possible to know what type of programming operation is present in the queue.

This flag is reset by hardware when all write, erase or option byte change operations have been executed and thus removed from the waiting queue(s). This bit cannot be forced to 0. It is reset after a deterministic time if no other operations are requested.

0: no write, erase or option byte change operations waiting in the operation queues
1: at least one write, erase or option byte change operation is waiting in the operation queue

Bit 1 **WBNE**: Write buffer not empty flag

WBNE flag is set when the embedded Flash memory is waiting for new data to complete the write buffer. In this state, the write buffer is not empty. WBNE is reset by hardware each time the write buffer is complete or the write buffer is emptied following one of the event below:

- the application software forces the write operation using FW bit in FLASH_CR
- the embedded Flash memory detects an error that involves data loss
- the application software has disabled write operations

This bit cannot be forced to 0. To reset it, clear the write buffer by performing any of the above listed actions, or send the missing data.

0: write buffer empty or full
1: write buffer waiting data to complete

Bit 0 **BSY**: Busy flag

BSY flag is set when an effective write, erase or option byte change operation is ongoing. It is not possible to know what type of operation is being executed.

BSY cannot be forced to 0. It is automatically reset by hardware every time a step in a write, erase or option byte change operation completes.

0: no programming, erase or option byte change operation being executed
1: programming, erase or option byte change operation being executed

4.9.6 FLASH clear control register (FLASH_CCR)

Address offset: 0x014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CLR_CRCDERR	CLR_CRCEND	CLR_DBECCERR	CLR_SNECCERR	CLR_RDSERR	CLR_RDPERR	CLR_OPERR	CLR_INCERR	Res.	CLR_STRBERR	CLR_PGSERR	CLR_WRPERR	CLR_EOP
			w	w	w	w	w	w	w	w		w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **CLR_CRCDERR**: CRCDERR flag clear bit

Setting this bit to 1 resets to 0 CRCDERR flag in FLASH_SR register.

Bit 27 **CLR_CRCEND**: CRCEND flag clear bit

Setting this bit to 1 resets to 0 CRCEND flag in FLASH_SR register.

Bit 26 **CLR_DBECCERR**: DBECCERR flag clear bit

Setting this bit to 1 resets to 0 DBECCERR flag in FLASH_SR register. If the SNECCERR flag of FLASH_SR register is set to 0, FLASH_ECC_FAR register is reset to 0 as well.

Bit 25 **CLR_SNECCERR**: SNECCERR flag clear bit

Setting this bit to 1 resets to 0 SNECCERR flag in FLASH_SR register. If the DBECCERR flag of FLASH_SR register is set to 0, FLASH_ECC_FAR register is reset to 0 as well.

Bit 24 **CLR_RDSERR**: RDSERR flag clear bit

Setting this bit to 1 resets to 0 RDSERR flag in FLASH_SR register.

Bit 23 **CLR_RDPERR**: RDPERR flag clear bit

Setting this bit to 1 resets to 0 RDPERR flag in FLASH_SR register.

Bit 22 **CLR_OPERR**: OPERR flag clear bit

Setting this bit to 1 resets to 0 OPERR flag in FLASH_SR register.

Bit 21 **CLR_INCERR**: INCERR flag clear bit

Setting this bit to 1 resets to 0 INCERR flag in FLASH_SR register.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CLR_STRBERR**: STRBERR flag clear bit

Setting this bit to 1 resets to 0 STRBERR flag in FLASH_SR register.

Bit 18 **CLR_PGSERR**: PGSERR flag clear bit

Setting this bit to 1 resets to 0 PGSERR flag in FLASH_SR register.

Bit 17 **CLR_WRPERR**: WRPERR flag clear bit

Setting this bit to 1 resets to 0 WRPERR flag in FLASH_SR register.

Bit 16 **CLR_EOP**: EOP flag clear bit

Setting this bit to 1 resets to 0 EOP flag in FLASH_SR register.

Bits 15:0 Reserved, must be kept at reset value.

4.9.7 FLASH option control register (FLASH_OPTCR)

Address offset: 0x018

Reset value: 0xX000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OPTCHANGEERRIE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPTSTART	OPTLOCK
														w	rs

Bit 31 Reserved, must be kept at reset value.

Bit 30 **OPTCHANGEERRIE**: Option byte change error interrupt enable bit

OPTCHANGEERRIE bit controls if an interrupt has to be generated when an error occurs during an option byte change.

0: no interrupt is generated when an error occurs during an option byte change

1: an interrupt is generated when and error occurs during an option byte change.

Bits 29:5 Reserved, must be kept at reset value.

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **OPTSTART**: Option byte start change option configuration bit

OPTSTART triggers an option byte change operation. The user can set OPTSTART only when the OPTLOCK bit is set to 0. The embedded Flash memory resets OPTSTART when the option byte change operation has been acknowledged.

The user application cannot modify any embedded Flash memory register until the option change operation has been completed.

Before setting this bit, the user has to write the required values in the FLASH_XXX_PRG registers. The FLASH_XXX_PRG registers will be locked until the option byte change operation has been executed in non-volatile memory.

It is not possible to start an option byte change operation if a CRC calculation is ongoing.

Trying to set OPTSTART when CRC_BUSY of FLASH_SR register is set has not effect; the option byte change does not start and no error is generated.

Bit 0 **OPTLOCK**: FLASH_OPTCR lock option configuration bit

The OPTLOCK bit locks the FLASH_OPTCR register as well as all _PRG registers. The correct write sequence to FLASH_OPTKEYR register unlocks this bit. If a wrong sequence is executed, or the unlock sequence to FLASH_OPTKEYR is performed twice, this bit remains locked until next system reset.

It is possible to set OPTLOCK by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When OPTLOCK changes from 0 to 1, the others bits of FLASH_OPTCR register do not change.

0: FLASH_OPTCR register unlocked

1: FLASH_OPTCR register locked.

4.9.8 FLASH option status register (FLASH_OPTSR_CUR)

Address offset: 0x01C

Reset value: 0xFFFF XXXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURITY	ST_RAM_SIZE[1:0]		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
	r	r								r	r	r	r	r	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP[7:0]								NRST_STOP_D1	NRST_STOP_D1	Res.	IWDG1_SW	BOR_LEV[1:0]		Res.	OPT_BUSY
r	r	r	r	r	r	r	r	r	r		r	r	r		r

- Bit 31 Reserved, must be kept at reset value.
- Bit 30 **OPTCHANGEERR**: Option byte change error flag
 OPTCHANGEERR flag indicates that an error occurred during an option byte change operation. When OPTCHANGEERR is set to 1, the option byte change operation did not successfully complete. An interrupt is generated when this flag is raised if the OPTCHANGEERRIE bit of FLASH_OPTCR register is set to 1.
 Writing 1 to CLR_OPTCHANGEERR of register FLASH_OPTCCR clears OPTCHANGEERR.
 0: no option byte change errors occurred
 1: one or more errors occurred during an option byte change operation.
- Bit 29 **IO_HSLV**: I/O high-speed at low-voltage status bit
 This bit indicates that the product operates below 2.5 V.
 0: Product working in the full voltage range, I/O speed optimization at low-voltage disabled
 1: Product operating below 2.5 V, I/O speed optimization at low-voltage feature allowed
- Bits 28:26 Reserved, must be kept at reset value.
- Bits 25:22 Reserved, must be kept at reset value.
- Bit 21 **SECURITY**: Security enable option status bit
 0: Security feature disabled
 1: Security feature enabled.
- Bits 20:19 **ST_RAM_SIZE[1:0]**: ST RAM size option status
 00: 2 Kbytes reserved to ST code
 01: 4 Kbytes reserved to ST code
 10: 8 Kbytes reserved to ST code
 11: 16 Kbytes reserved to ST code
*Note: This bitfield is effective only when the security is enabled (SECURITY = 1).
 The whole DTCM RAM is always available for the application whatever ST_RAM_SIZE option byte configuration.*
- Bit 18 **IWDG_FZ_SDBY**: IWDG Standby mode freeze option status bit
 When set the independent watchdog IWDG1 is frozen in system Standby mode.
 0: Independent watchdog frozen in Standby mode
 1: Independent watchdog keep running in Standby mode.
- Bit 17 **IWDG_FZ_STOP**: IWDG Stop mode freeze option status bit
 When set the independent watchdog IWDG1 is in system Stop mode.
 0: Independent watchdog frozen in system Stop mode
 1: Independent watchdog keep running in system Stop mode.
- Bit 16 Reserved, must be kept at reset value.
- Bits 15:8 **RDP[7:0]**: Readout protection level option status byte
 0xAA: global readout protection level 0
 0xCC: global readout protection level 2
 others values: global readout protection level 1.
- Bit 7 **NRST_STDY_D1**: D1 domain DStandby entry reset option status bit
 0: a reset is generated when entering DStandby mode on D1 domain
 1: no reset generated when entering DStandby mode on D1 domain
- Bit 6 **NRST_STOP_D1**: D1 domain DStop entry reset option status bit
 0: a reset is generated when entering DStop mode on D1 domain
 1: no reset generated when entering DStop mode on D1 domain.

Bit 5 Reserved, must be kept at reset value.

Bit 4 **IWDG1_SW**: IWDG1 control mode option status bit

1: IWDG1 watchdog is controlled by software

0: IWDG1 watchdog is controlled by hardware.

Bits 3:2 **BOR_LEV[1:0]**: Brownout level option status bit

These bits reflect the power level that generates a system reset.

00: Brownout reset threshold 0 (V_{BOR0})

01: Brownout reset threshold 1 (V_{BOR1})

10: Brownout reset threshold 2 (V_{BOR2})

11: Brownout reset threshold 3 (V_{BOR3})

Bit 1 Reserved, must be kept at reset value.

Bit 0 **OPT_BUSY**: Option byte change ongoing flag

OPT_BUSY indicates if an option byte change is ongoing. When this bit is set to 1, the embedded Flash memory is performing an option change and it is not possible to modify any embedded Flash memory register.

0: no option byte change ongoing

1: an option byte change ongoing and all write accesses to Flash registers are blocked until the option byte change completes.

4.9.9 FLASH option status register (FLASH_OPTSR_PRG)

Address offset: 0x020

Reset value: 0xFFFF XXXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register is used to program values in corresponding option bits. Values after reset reflect the current values of the corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	IO_HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURITY	ST_RAM_SIZE[1:0]		IWDG_FZ_SDBY	IWDG_FZ_STOP	Res.
		rw								rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP[7:0]								NRST_STOP_D1	NRST_STOP_D1	Res.	IWDG1_SW	BOR_LEV[1:0]		Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw		

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **IO_HSLV**: I/O high-speed at low-voltage configuration bit

This bit indicates that the product operates below 2.5 V.

0: Product working in the full voltage range, I/O speed optimization at low-voltage disabled

1: Product operating below 2.5 V, I/O speed optimization at low-voltage feature allowed

Bits 28:26 Reserved, must be kept at reset value.

Bits 25:22 Reserved, must be kept at reset value.

Bit 21 **SECURITY**: Security enable option configuration bit

The SECURITY option bit enables the secure access mode of the device during an option byte change. The change will be taken into account at next power-on reset. Once it is enabled, the security feature can be disabled if no areas are protected by PCROP or Secure access mode. If there are secure-only or PCROP protected areas, perform a regression level (from level 1 to 0) and set all the bits to unprotect secure-only areas and PCROP areas.

0: Security feature disabled

1: Security feature enabled.

Bits 20:19 **ST_RAM_SIZE[1:0]**: ST RAM size option configuration bits

00: 2 Kbytes reserved to ST code

01: 4 Kbytes reserved to ST code

10: 8 Kbytes reserved to ST code

11: 16 Kbytes reserved to ST code

Note: This bitfield is effective only when the security is enabled (SECURITY = 1).

The whole DTCM RAM is always available for the application whatever ST_RAM_SIZE option byte configuration.

Bit 18 **IWDG_FZ_SDBY**: IWDG Standby mode freeze option configuration bit

This option bit is used to freeze or not the independent watchdog IWDG1 in system Standby mode.

0: Independent watchdog frozen in Standby mode

1: Independent watchdog keep running in Standby mode.

Bit 17 **IWDG_FZ_STOP**: IWDG Stop mode freeze option configuration bit

This option bit is used to freeze or not the independent watchdog IWDG1 in system Stop mode.

0: Independent watchdog frozen in system Stop mode

1: Independent watchdog keep running in system Stop mode.

Bit 16 Reserved, must be kept at reset value.

Bits 15:8 **RDP[7:0]**: Readout protection level option configuration bits

RDP bits are used to change the readout protection level. This change is possible only when the current protection level is different from level 2. The possible configurations are:

0xAA: global readout protection level 0

0xCC: global readout protection level 2

others values: global readout protection level 1.

Bit 7 **NRST_STDY_D1**: D1 domain DStandby entry reset option configuration bit

0: a reset is generated when entering DStandby mode on D1 domain.

1: no reset generated when entering DStandby mode on D1 domain

Bit 6 **NRST_STOP_D1**: D1 domain DStop entry reset option configuration bit

0: a reset is generated when entering DStop mode on D1 domain.

1: no reset generated when entering DStop mode on D1 domain.

Bit 5 Reserved, must be kept at reset value.

Bit 4 **IWDG1_SW**: IWDG1 control mode option configuration bit

IWDG1_SW option bit is used to select if IWDG1 independent watchdog is controlled by hardware or by software.

1: IWDG1 watchdog is controlled by software.

0: IWDG1 watchdog is controlled by hardware.

Bits 3:2 **BOR_LEV[1:0]**: Brownout level option configuration bit

These option bits are used to define the power level that generates a system reset.

00 and 11: the reset level is set to 2.1 V

01: the reset is set to 2.4 V

10: the reset is set to 2.7 V

Bits 1:0 Reserved, must be kept at reset value.

4.9.10 FLASH option clear control register (FLASH_OPTCCR)

Address offset: 0x024

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	CLR_OPTCHANGEERR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	w														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bit 31 Reserved, must be kept at reset value.

Bit 30 **CLR_OPTCHANGEERR**: OPTCHANGEERR reset bit

This bit is used to reset the OPTCHANGEERR flag in FLASH_OPTSR_CUR or FLASH_OPTSR2_CUR register. FLASH_OPTCCR is write-only.

It is reset by programming it to 1.

Bits 29:0 Reserved, must be kept at reset value.

4.9.11 FLASH protection address (FLASH_PRAR_CUR)

Address offset: 0x028

Reset value: 0xFFFF 0XXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP	Res.	Res.	Res.	PROT_AREA_END[11:0]											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMEP**: PCROP protected erase enable option status bit

If DMEP is set to 1, the PCROP protected area is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END[11:0]**: PCROP area end status bits

These bits contain the last 256-byte block of the PCROP area.

If this address is equal to PROT_AREA_START, the whole bank is PCROP protected.

If this address is lower than PROT_AREA_START, no protection is set.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START[11:0]**: PCROP area start status bits

These bits contain the first 256-byte block of the PCROP area.

If this address is equal to PROT_AREA_END, the whole bank is PCROP protected.

If this address is higher than PROT_AREA_END, no protection is set.

4.9.12 FLASH protection address (FLASH_PRAR_PRG)

Address offset: 0x02C

Reset value: 0xFFFF 0XXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMEP	Res.	Res.	Res.	PROT_AREA_END[11:0]											
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PROT_AREA_START[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **DMEP**: PCROP protected erase enable option configuration bit

If DMEP is set to 1, the PCROP protected area is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **PROT_AREA_END[11:0]**: PCROP area end configuration bits

These bits contain the last 256-byte block of the PCROP area.

If this address is equal to PROT_AREA_START, the whole bank is PCROP protected.

If this address is lower than PROT_AREA_START, no protection is set.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **PROT_AREA_START[11:0]**: PCROP area start configuration bits

These bits contain the first 256-byte block of the PCROP area.

If this address is equal to PROT_AREA_END, the whole bank is PCROP protected.

If this address is higher than PROT_AREA_END, no protection is set.

4.9.13 FLASH secure address (FLASH_SCAR_CUR)

Address offset: 0x030

Reset value: 0xFFFF 0XXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMES	Res.	Res.	Res.	SEC_AREA_END[11:0]											
r				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **DMES**: secure access protected erase enable option status bit

If DMES is set to 1, the secure access only area is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END[11:0]**: secure-only area end status bits

These bits contain the last 256-byte block of the secure-only area.

If this address is equal to SEC_AREA_START, the whole bank is secure access only.

If this address is lower than SEC_AREA_START, no protection is set.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START[11:0]**: secure-only area start status bits

These bits contain the first 256 bytes of block of the secure-only area.

If this address is equal to SEC_AREA_END, the whole bank is secure access only.

If this address is higher than SEC_AREA_END, no protection is set.

4.9.14 FLASH secure address (FLASH_SCAR_PRG)

Address offset: 0x034

Reset value: 0xFFFF 0XXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DMES	Res.	Res.	Res.	SEC_AREA_END[11:0]											
rw				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	SEC_AREA_START[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **DMES**: Secure access protected erase enable option configuration bit

If DMES is set to 1, the secure access only area is erased when a protection level regression (change from level 1 to 0) or a bank erase with protection removal occurs.

Bits 30:28 Reserved, must be kept at reset value.

Bits 27:16 **SEC_AREA_END[11:0]**: Secure-only area end configuration bits

These bits contain the last block of 256 bytes of the secure-only area.

If this address is equal to SEC_AREA_START, the whole bank is secure access only.

If this address is lower than SEC_AREA_START, no protection is set.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **SEC_AREA_START[11:0]**: Secure-only area start configuration bits

These bits contain the first block of 256 bytes of the secure-only area.

If this address is equal to SEC_AREA_END, the whole bank is secure access only.

If this address is higher than SEC_AREA_END, no protection is set.

4.9.15 FLASH write sector protection (FLASH_WPSN_CUR)

Address offset: 0x038

Reset value: 0x0000 00XX

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPS7	WRPS6	WRPS5	WRPS4	WRPS3	WRPS2	WRPS1	WRPS0
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn**: Sector n write protection option status bit (n = 7 to 0)

Each FLASH_WPSN_CUR bit reflects the write protection status of the corresponding sector (0: sector is write protected; 1: sector is not write protected)

4.9.16 FLASH write sector protection (FLASH_WPSN_PRG)

Address offset: 0x03C

Reset value: 0x0000 00XX

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPS7	WRPS6	WRPS5	WRPS4	WRPS3	WRPS2	WRPS1	WRPS0
								rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WRPSn**: Sector n write protection option status bit (n = 7 to 0)

Setting WRPSn bit to 0 write protects the corresponding sector (0: sector is write protected; 1: sector is not write protected).

4.9.17 FLASH register boot address for Arm® Cortex®-M7 core (FLASH_BOOT_CUR)

Address offset: 0x040

Reset value: 0xFFFF XXXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM_ADD1[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM_ADD0[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 **BOOT_CM_ADD1[15:0]**: Arm® Cortex®-M7 boot address 1

These bits reflect the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM_ADD0[15:0]**: Arm® Cortex®-M7 boot address 0

These bits reflect the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is low.

4.9.18 FLASH register boot address for Arm® Cortex®-M7 core (FLASH_BOOT_PRG)

Address offset: 0x044

Reset value: 0xFFFF XXXX

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register is used to program values in corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BOOT_CM_ADD1[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT_CM_ADD0[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:16 **BOOT_CM_ADD1[15:0]**: Arm® Cortex®-M7 boot address 1 configuration

These bits allow configuring the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is high.

Bits 15:0 **BOOT_CM_ADD0[15:0]**: Arm® Cortex®-M7 boot address 0 configuration

These bits allow configuring the MSB of the Arm® Cortex®-M7 boot address when the BOOT pin is low.

4.9.19 FLASH CRC control register (FLASH_CRCCR)

Address offset: 0x050

Reset value: 0x001C 0000

This register can be modified only if CRC_EN bit is set to 1 in FLASH_CR register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST[1:0]		Res.	Res.	CLEAN_CRC	START_CRC
									w	rW	rW			rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.	CRC_SECT[2:0]		
					w	w	rW						rW	rW	rW

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **ALL_BANK**: CRC select bit

When ALL_BANK is set to 1, all bank user sectors are added to list of sectors on which the CRC is calculated.

Bits 21:20 **CRC_BURST[1:0]**: CRC burst size

CRC_BURST bits set the size of the bursts that are generated by the CRC calculation unit.

00: every burst has a size of 4 Flash words (256-bit)

01: every burst has a size of 16 Flash words (256-bit)

10: every burst has a size of 64 Flash words (256-bit)

11: every burst has a size of 256 Flash words (256-bit)

Bits 19:18 Reserved, must be kept at reset value.

Bit 17 **CLEAN_CRC**: CRC clear bit

Setting CLEAN_CRC to 1 clears the current CRC result stored in the FLASH_CRCDATAR register.

Bit 16 **START_CRC**: CRC start bit

START_CRC bit triggers a CRC calculation using the current configuration. No CRC calculation can be launched when an option byte change operation is ongoing because all write accesses to embedded Flash memory registers are put on hold until the option byte change operation has completed.

Bits 15:11 Reserved, must be kept at reset value.

Bit 10 **CLEAN_SECT**: CRC sector list clear bit

Setting CLEAN_SECT to 1 clears the list of sectors on which the CRC is calculated.

Bit 9 **ADD_SECT**: CRC sector select bit

Setting ADD_SECT to 1 adds the sector whose number is CRC_SECT to the list of sectors on which the CRC is calculated.

Bit 8 **CRC_BY_SECT**: CRC sector mode select bit

When CRC_BY_SECT is set to 1, the CRC calculation is performed at sector level, on the sectors present in the list of sectors. To add a sector to this list, use ADD_SECT and CRC_SECT bits. To clean the list, use CLEAN_SECT bit.

When CRC_BY_SECT is reset to 0, the CRC calculation is performed on all addresses between CRC_START_ADDR and CRC_END_ADDR.

Bits 7:3 Reserved, must be kept at reset value.

Bits 2:0 **CRC_SECT[2:0]**: CRC sector number

CRC_SECT is used to select one or more user Flash sectors to be added to the list of sectors on which the CRC is calculated. The CRC can be computed either between two addresses (using registers FLASH_CRCSADDR and FLASH_CRCEADDR) or on a list of sectors. If this latter option is selected, it is possible to add a sector to the list of sectors by programming the sector number in CRC_SECT and then setting ADD_SECT to 1.

The list of sectors can be erased either by setting CLEAN_SECT bit or by disabling the CRC computation. CRC_SECT can be set only when CRC_EN of FLASH_CR register is set to 1.

000: sector 0

001: sector 1

...

111: sector 7

4.9.20 FLASH CRC start address register (FLASH_CRCSADDR)

Address offset: 0x054

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_START_ADDR[15:2]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_START_ADDR[19:2]**: CRC start address

CRC_START_ADDR is used when CRC_BY_SECT is set to 0. It must be programmed to the start address of the bank memory area on which the CRC calculation is performed.

Bits 1:0 Reserved, must be kept at reset value.

4.9.21 FLASH CRC end address register (FLASH_CRCEADDR)

Address offset: 0x058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[19:16]			
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_END_ADDR[15:2]													Res.	Res.	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		

Bits 31:20 Reserved, must be kept at reset value.

Bits 19:2 **CRC_END_ADDR[19:2]**: CRC end address

CRC_END_ADDR is used when CRC_BY_SECT is set to 0. It must be programmed to the end address of the bank memory area on which the CRC calculation is performed

Bits 1:0 Reserved, must be kept at reset value.

4.9.22 FLASH CRC data register (FLASH_CRCDATAR)

Address offset: 0x05C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CRC_DATA[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_DATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **CRC_DATA[31:0]**: CRC result

CRC_DATA bits contain the result of the last CRC calculation.

4.9.23 FLASH ECC fail address (FLASH_ECC_FAR)

Address offset: 0x060

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FAIL_ECC_ADDR[14:0]														
	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:15 Reserved, must be kept at reset value.

Bits 14:0 **FAIL_ECC_ADDR[14:0]**: ECC error address

When an ECC error occurs (both for single correction or double detection) during a read operation, the FAIL_ECC_ADDR bitfield contains the address that generated the error.

FAIL_ECC_ADDR is reset when the flag error in the FLASH_SR register (CLR_SNECCERR or CLR_DBECCERR) is reset.

The embedded Flash memory programs the address in this register only when no ECC error flags are set. This means that only the first address that generated an ECC error is saved.

The address in FAIL_ECC_ADDR is relative to the Flash area where the error occurred (user Flash, system Flash).

Fail address = FAIL_ECC_ADDR[14:0] * 32 + Flash bank offset

4.9.24 FLASH option status register 2 (FLASH_OPTSR2_CUR)

Address offset: 0x070

Reset value: 0x0000 000X

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This read-only register reflects the current values of corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CPUFREQ_BOOST	TCM_AXI_SHARED[1:0]	
													r	r	r

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **CPUFREQ_BOOST**: CPU frequency boost status bit

This bit indicates whether the CPU frequency can be boosted or not. When it is set, the ECC on ITCM and DTCM are no more used.

Bits 1:0 **TCM_AXI_SHARED[1:0]**: TCM RAM sharing status bit

This bitfield contains the ITCM memory size and the AXI system RAM.

00: 64-Kbyte ITCM / 320 Kbyte system AXI

01: 128-Kbyte ITCM / 256-Kbyte system AXI

10: 192-Kbyte ITCM / 192-Kbyte system AXI

11: 256-Kbyte ITCM / 128-Kbyte system AXI

4.9.25 FLASH option status register 2 (FLASH_OPTSR2_PRG)

Address offset: 0x074

Reset value: 0x0000 000X

Refer to [Table 18: Option byte organization](#) for details on the reset value.

This register is used to program values in corresponding option bits. Values after reset reflects the current values of the corresponding option bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CPUFREQ_BOOST	TCM_AXI_SHARED[1:0]	
													rw	rw	rw

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **CPUFREQ_BOOST**: CPU frequency boost status bit

This bit configures whether the CPU frequency can be boosted or not. When it is set, the ECC on ITCM and DTCM are no more used.

Bits 1:0 **TCM_AXI_SHARED[1:0]**: TCM RAM sharing status bit

This bitfield configures the ITCM memory size and the AXI system RAM.

00: 64-Kbyte ITCM / 320 Kbyte system AXI

01: 128-Kbyte ITCM / 256-Kbyte system AXI

10: 192-Kbyte ITCM / 192-Kbyte system AXI

11: 256-Kbyte ITCM / 128-Kbyte system AXI

4.10 FLASH register map and reset values

Table 25. Register map and reset value table

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x000	FLASH_ACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRHIGHREQ	1	1	0	1	1	1
	0x00000037																																	
0x004	FLASH_KEYR	KEYKEYRR																																
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008	FLASH_OPTKEYR	OPTKEYR																																
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	FLASH_CR	Res.	Res.	Res.	CRCRDERRIE	CRCENDIE	DBECCERRIE	SNECCERRIE	RDSERRIE	RDPERRIE	OPERRIE	INCERRIE	Res.	STRBERRIE	PGSERRIE	WRPERRIE	EOPIE	CRC_EN	Res.	Res.	Res.	SNB				START	FW	PSIZE	BER	SER	PG	LOCK		
	0x00000031				0	0	0	0	0	0	0	0		0	0	0	0	0				0	0	0	0	0	0	0	1	1	0	0	0	1
0x010	FLASH_SR	Res.	Res.	Res.	CRCRDERR	CRCEND	DBECCERR	SNECCERR	RDSERR	RDPERR	OPERRIE	INCERR	Res.	STRBERR	PGSERR	WRPERR	EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BUSY	QW	WBNE	BSY
	0x00000000				0	0	0	0	0	0	0	0		0	0	0	0													0	0	0	0	
0x014	FLASH_CCR	Res.	Res.	Res.	CLR_CRCRDERR	CLR_CRCEND	CLR_DBECCERR	CLR_SNECCERR	CLR_RDSERR	CLR_RDPERR	CLR_OPERRIE	CLR_INCERR	Res.	CLR_STRBERR	CLR_PGSERR	CLR_WRPERR	CLR_EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0x00000000					0	0	0	0	0	0	0		0	0	0	0																	
0x018	FLASH_OPTCR	Res.	OPTCHANGEERRIE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0x00000001	0																															0	1
0x01C	FLASH_OPTSR_CUR	Res.	OPTCHANGEERR	IO_HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURITY	ST_RAM_SIZE	IWDG_FZ_SDBY	IWDG_FZ_STOP	RDP[7:0]	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
	0xFFFF XXXX	X	X																															

Table 25. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x020	FLASH_OPTSR_PRG	Res.	Res.	IO_HSLV	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECURITY	ST_RAM_SIZE	19	18	IWDG_FZ_SDBY	IWDG_FZ_STOP	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	0XXXXX XXXX			X								X	X	X	X	X	X	Res.	X	X	X	X	X	X	X	X	NRST_STBY_D1	NRST_STOP_D1	Res.	IWDG1_SW	BOR_LEV		Res.	Res.
0x024	FLASH_OPTCCR	Res.	CLR_OPTCHANGEERR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0x00000000	0																																
0x028	FLASH_PRAR_CUR	DMEP	Res.	Res.	Res.	PROT_AREA_END[11:0]												Res.	Res.	Res.	Res.	PROT_AREA_START[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X	X					X	X	X	X	X	X	X	X	X	X	X	X
0x02C	FLASH_PRAR_PRG	DMEP	Res.	Res.	Res.	PROT_AREA_END[11:0]												Res.	Res.	Res.	Res.	PROT_AREA_START[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X	X	Res.					X	X	X	X	X	X	X	X	X	X	X
0x030	FLASH_SCAR_CUR	DMES	Res.	Res.	Res.	SEC_AREA_END[11:0]												Res.	Res.	Res.	Res.	SEC_AREA_START[11:0]												
	0XXXXX 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X	X	Res.					X	X	X	X	X	X	X	X	X	X	X

Table 25. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x034	FLASH_SCAR_PRG	DMES	Res.	Res.	Res.	SEC_AREA_END[11:0]												Res.	Res.	Res.	Res.	SEC_AREA_START[11:0]											
	0xFFFF 0XXX	X				X	X	X	X	X	X	X	X	X	X	X	X						X	X	X	X	X	X	X	X	X	X	X
0x038	FLASH_WPSN_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn											
	0x0000 00XX	0																									X	X	X	X	X	X	X
0x03C	FLASH_WPSN_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRPSn											
	0x0000 00XX																										X	X	X	X	X	X	X
0x040	FLASH_BOOT_CUR	BOOT_CM_ADD1[15:0]															BOOT_CM_ADD0[15:0]																
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x044	FLASH_BOOT_PRG	BOOT_CM_ADD1[15:0]															BOOT_CM_ADD0[15:0]																
	0XXXXX XXXX	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x050	FLASH_CRCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALL_BANK	CRC_BURST	Res.	Res.	CLEAN_CRC	START_CRC	Res.	Res.	Res.	Res.	Res.	Res.	CLEAN_SECT	ADD_SECT	CRC_BY_SECT	Res.	Res.	Res.	Res.	Res.	CRC_SECT		
	0x001C0000										0	0	1		0	0							0	0	0						0	0	0
0x054	FLASH_CRCSADDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_START_ADDR[31:0]																	Res.	Res.	
	0x00000000													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x058	FLASH_CRCEADDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_END_ADDR[31:0]																	Res.	Res.	
	0x00000000													0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x05C	FLASH_CRCDATAR	CRC_DATA[31:0]																															
	0x00000000	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x060	FLASH_ECC_FAR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FAIL_ECC_ADDR[14:0]														
	0x00000000																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x070	FLASH_OPTSR2_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CPUFREQ_BOOST	TCM_AXI_SHARED[1:0]	
	0XXXXX 0XXX																																

Table 25. Register map and reset value table (continued)

Offset	Register name reset	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x074	FLASH_OPTSR2 _PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	0xFFFF 0XXX																																

Refer to [Section 2.3 on page 131](#) for the register boundary addresses.